

Module 1

Introduction to Java and OOP Concepts

Introduction to Java

Java was conceived by James Gosling, Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan at Sun Microsystems, Inc. in 1991. It took 18 months to develop the first working version. This language was initially called “Oak,” but was renamed “Java” in 1995. Between the initial implementation of Oak in the fall of 1992 and the public announcement of Java in the spring of 1995, many more people contributed to the design and evolution of the language. Bill Joy, Arthur van Hoff, Jonathan Payne, Frank Yellin, and Tim Lindholm were key contributors to the maturing of the original prototype.

Somewhat surprisingly, the original impetus for Java was not the Internet! Instead, the primary motivation was the need for a platform-independent (that is, architecture-neutral) language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls. As you can probably guess, many different types of CPUs are used as controllers. The trouble with C and C++ (and most other languages) is that they are designed to be compiled for a specific target. Although it is possible to compile a C++ program for just about any type of CPU, to do so requires a full C++ compiler targeted for that CPU. The problem is that compilers are expensive and time-consuming to create. An easier—and more cost-efficient—solution was needed. In an attempt to find such a solution, Gosling and others began work on a portable, platform-independent language that could be used to produce code that would run on a variety of CPUs under differing environments.

This effort ultimately led to the creation of Java. About the time that the details of Java were being worked out, a second, and ultimately more important, factor was emerging that would play a crucial role in the future of Java. This second force was, of course, the World Wide Web. Had the Web not taken shape at about the same time that Java was being implemented, Java might have remained a useful but obscure language for programming consumer electronics. However, with the emergence of the World Wide Web, Java was propelled to the forefront of computer language design, because the Web, too, demanded portable programs.

Most programmers learn early in their careers that portable programs are as elusive as they are desirable. While the quest for a way to create efficient, portable (platform-independent) programs is nearly as old as the discipline of

programming itself, it had taken a back seat to other, more pressing problems.

Further, because (at that time) much of the computer world had divided itself

into the three competing camps of Intel, Macintosh, and UNIX, most programmers stayed within their fortified boundaries, and the urgent need for portable code was reduced. However, with the advent of the Internet and the Web, the old problem of portability returned with a vengeance. After all, the Internet consists of a diverse, distributed universe populated with various types of computers, operating systems, and CPUs. Even though many kinds of platforms are attached to the Internet, users would like them all to be able to run the same program. What was once an irritating but low-priority problem had become a high-profile necessity. By 1993, it became obvious to members of the Java design team that the problems of portability frequently encountered when creating code for embedded controllers are also found when attempting to create code for the Internet. In fact, the same problem that Java was initially designed to solve on a small scale could also be applied to the Internet on a large scale. This realization caused the focus of Java to switch from consumer electronics to Internet

programming. So, while the desire for an architecture-neutral programming language provided the initial spark, the Internet ultimately led to Java's large scale success.

As mentioned earlier, Java derives much of its character from C and C++. This is by intent. The Java designers knew that using the familiar syntax of C and echoing the object-oriented features of C++ would make their language appealing to the legions of experienced C/C++ programmers. In addition to the surface similarities, Java shares some of the other attributes that helped make C and C++ successful. First, Java was designed, tested, and refined by real, working programmers. It is a language grounded in the needs and experiences of the people who devised it. Thus, Java is a programmer's language. Second, Java is cohesive and logically consistent. Third, except for those constraints imposed by the Internet environment, Java gives you, the programmer, full control. If you program well, your programs reflect it. If you program poorly, your programs reflect that, too. Put differently, Java is not a language with training wheels. It is a language for professional programmers.

Because of the similarities between Java and C++, it is tempting to think of Java as simply the "Internet version of C++." However, to do so would be a large mistake. Java has significant practical and philosophical differences. While it is true that Java was influenced by C++, it is not an enhanced version of C++. For example, Java is neither upwardly nor downwardly compatible with C++. Of course, the similarities with C++ are significant, and if you are a C++ programmer, then you will feel right at home with Java. One other point: Java was not designed to replace C++. Java was designed to solve a certain set of Problems. C++ was designed to solve a different set of problems. Both will coexist for many years to come. As mentioned at the start of this chapter, computer languages evolve for two Reasons: to adapt to changes in environment and to implement advances in the art of programming. The environmental change that prompted Java was the need for platform-independent programs destined for distribution on the Internet. However, Java also embodies changes in the way that people approach the Writing of programs. For example, Java enhanced and refined the object-oriented paradigm used by C++, added integrated support for multithreading, and provided a library that simplified Internet access. In the final analysis, though, it was not the individual features of Java that made it so remarkable. Rather, it was the language as a whole. Java was the perfect response to the demands of the then newly emerging, highly distributed computing universe. Java was to Internet programming what C was to system programming: a revolutionary force that changed the world.

Object Oriented Programming:

- ☐ Object oriented programming (OOP) is the core of Java programming.
- ☐ Java is a general purpose, object- oriented programming language developed by Sun Microsystems. It was invented by James Gosling and his team and was initially called as Oak.
- ☐ The most important feature that made Java very popular was the —Platform- Independent approach.
- ☐ It was the first programming language that did not tie-up with any particular operating system(or hardware) rather Java programs can be executed anywhere and on any system.
- ☐ Java was designed for the development of the software for consumer electronic devices like TVs, VCRs, etc.

Two Paradigms:

- ☐ Every program contains 2 components code and data.
- ☐ Two approaches are there to solve the problem and in program writing: Procedure oriented and object oriented.

Procedure Oriented:

- Procedure oriented programs are written based on —whats happening around, where the code acts on data. Ex: C etc
- Problems increases in procedure oriented as the program grows larger and more complex.

Object Oriented:

- Object oriented programs are written based on —Who is being affected around, which manages the increasing complexity.
- It organises program around data and well defined interfaces of that data.
- Characterised as data controlling access to code. Ex: C++, JAVA, Small Talk etc

The Three OOP Principles:

The three important principles of OOP are:

- Encapsulation
- Inheritance
- Polymorphism

Encapsulation:

- Encapsulation is the mechanism that binds together code and data it manipulates, and keeps both safe from outside interference and misuse.
- In Java the basis of encapsulation is the class. A class defines the state and behavior(data & code) that will be shared by set of objects.
- Each object contains the structure and behavior defined by the class. The data defined by the class are called instance variables(member variables), the code that operates on that data are called methods(member functions).

Inheritance:

- Inheritance is the process by which one object acquires the properties of another object. This is important as it supports the concept of hierarchical classification.
- By the use of inheritance, a class has to define only those qualities that make it unique. The general qualities can ber derived from the parent class or base class.
- Ex: A child inheriting properties from parents.

Polymorphism

- Polymorphism (meaning many forms) is a feature that allows one interface to be used for a general class of actions. The specific action determined by the exact nature of the situation. This concept is often expressed as “ one interface, multiple methods”.
- Ex: “+” can be used for addition of 2 numbers and also concatenation of 2 strings.
`System.out.println(2+4); // outputs 6 as answer`

`System.out.println(“Hello” + “Gautham”); // outputs Hello Gautham as answer` Apart from this the additional features include:

Object:

- ☐ An object can be any real world entity.
- ☐ Ex: an animal, bank, human, box, fan etc
- ☐ An object is a software bundle of related state and behavior.
- ☐ An object is an instance of class.

Class:

- ☐ A class is a blueprint or prototype from which objects are created.
- ☐ Its just a template for an object, which describes an object.
- ☐ Ex: a class describes how an animal looks like.
- ☐ A class is a user defined data type.

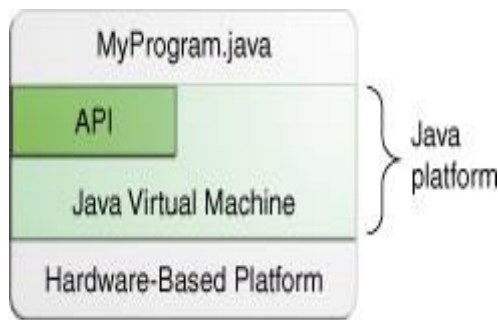
Abstraction:

- ☐ Data abstraction refers to providing only essential information to the outside world and hiding their background details i.e., to represent the needed information in program without presenting the details.
- ☐ Ex: a database system hides certain details of how data is stored and created and maintained.

I. Java Platform & the Creation of Java

Any hardware or software environment in which a program runs is known as a platform. Since Java has its own runtime environment (**JRE-Java Runtime Environment**) and API, it is called platform.

The Java platform has two components – API and Java Virtual Machine



The Java Virtual Machine :

It is an abstract machine and it is the base for the Java platform and is ported onto various hardware- based platforms.

The Java Application Programming Interface (API) :

The API is a large collection of ready-made software components that provide many useful capabilities. It is grouped into libraries of related classes and interfaces; these libraries are known as *packages*.

Java was originally developed to develop a language for digital devices such as set-top boxes and consumer electronic devices such as microwave ovens and remote controls.

Java was developed by **James Gosling**, Patrick Naughton, Mike Sheridan at *Sun Microsystems Inc.* in 1991. It took 18 months to develop the first working version.

The initial name was **Oak** but it was renamed to **Java** in 1995.

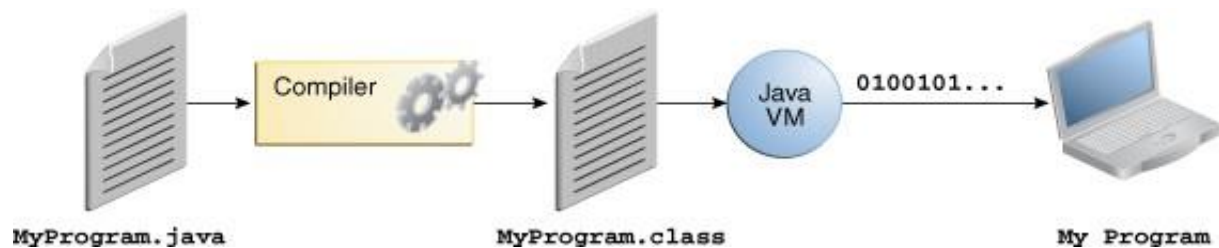
Primary Motivation(s) for Java creation:

1. The primary motivation was the need for a **platform-independent (architecture- neutral/portable)** language that could be used to create software to be embedded in various consumer electronic devices, such as microwave ovens and remote controls.
2. The second important factor is the **World Wide Web / Internet**.
 1. Java is used to develop web based programs / Internet programming.
 2. The Internet consists of a diverse, distributed universe populated with various types of computers, operating systems, and CPUs.

Platform Independence or Architecture Neutral or Portable:

(Write Once, Run Anywhere)

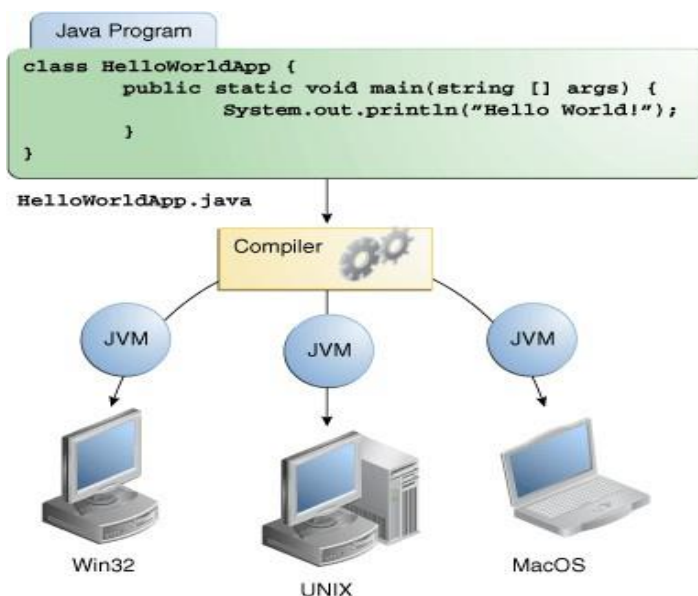
- a) Java is a Platform Independent language. Java sometimes called as “*Write Once, Run Anywhere (WORA)*” language.
- b) Java achieves platform independence through **bytecode** and **JVM**.



Java Program Execution:

1. In the Java programming language, all source code is first written in **plain text files** ending with the **.java** extension.
2. Those source files are then **compiled** into **.class files** by the **javac** compiler.
3. A **.class** file does not contain code that is native to your processor; it instead contains **bytecodes** — the machine language of the **Java Virtual Machine (Java VM)**.
4. The **java** launcher tool then runs your application with an instance of the Java Virtual Machine.
5. **Through the JVM**, the same application is capable of running on multiple platforms.

Java's magic: bytecode



Bytecode:

Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system, which is called the Java Virtual Machine (JVM).

Java is platform independent but JVM is platform dependent:

JVM is platform independent because JVM is designed to each platform separately.

1. Translating a Java program into bytecode makes it much easier to run a program in a wide variety of environments because only the JVM needs to be implemented for each platform.
2. Although the details of the JVM will differ from platform to platform, all understand the same *Java bytecode*.
3. A Java program is executed by the JVM also helps to make it secure. Because the JVM is in control, it can contain the program and prevent it from generating side effects outside of the system.

II. JAVA Buzzwords / Features

The prime reason behind creation of Java was to bring **portability** and **security** feature into a computer language. Beside these two major features, there were many other features that played an important role in Java language creation. Those features are:

1. Simple:

- Java is easy to learn and its syntax is quite simple, clean and easy to understand.
- It is simple because, Syntax is based on C and C++.
- Java removed pointers and operator overloading concepts.

2. Object oriented:

- In Java, everything is an object which has some data and behavior.
- The object model in Java is simple and easy to extend, while primitive types, such as integers, are kept as high-performance non-objects.

a) Java supports all the OOP features:

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

3. Distributed: Java is designed for the distributed environment of the Internet because it handles TCP/IP protocols. Java has features that enables a program to invoke methods across a network.

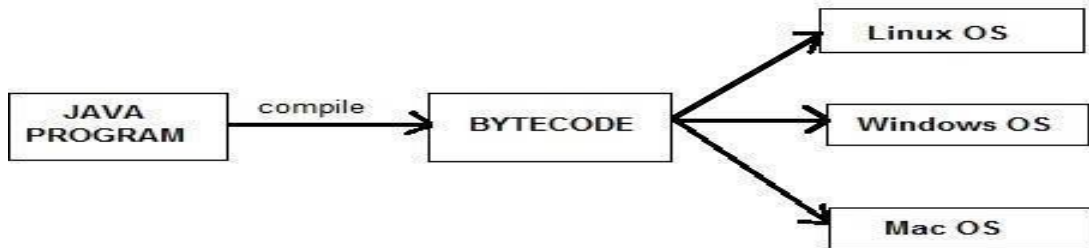
4. Multithreaded: With java's multi-threaded feature it is possible to write programs that can do many tasks simultaneously. This design feature allows developers to construct smoothly running interactive applications. Benefit of multithreading is that it utilizes same memory and other resources to execute multiple threads at the same time.

5. Dynamic: Java programs carry with them substantial amounts of *run-time type information* that is used to verify and resolve accesses to objects at *run time*. This makes it possible to dynamically link code in a safe and expedient manner. This is crucial to the robustness of the Java environment,

in which small fragments of bytecode may be dynamically updated on a running system.

6. Architecture Neutral (Platform Independent):

- C and C++ are platform dependency languages, But Java is platform independent language.
- Java designers goal was “**Write once; run anywhere, any time, forever**”(WORA). Java programs written in one operating system can able to run on any operating system.
- Java compiler generates an architecture-neutral bytecode which makes the compiled code to be executable on many processors with the presence of Java run time system (JVM).



7. Portable: Java is a portable language means, we can carry the same Java bytecode to any platform.

8. Interpreted: Java’s bytecode is interpreted by Java Virtual Machine (JVM). Java’s bytecode was carefully designed so that it will be easily translated into Native code.

9. High Performance: Java is faster than traditional interpretation since byte code was carefully designed so that it would be easy to translate directly into native machine code for very high performance by using a just-in-time compiler (JIT).

10. Robust: The ability to create robust programs was given a high priority in the design of Java.

Some of the reasons for Java is called as a **robust** language:

i. Strongly Typed Language

a. Java checks the code at compile time.

ii. Automatic Memory Management (Garbage collection)

a. In C/C++, the programmer will manually allocate and free all dynamic memory. This sometimes leads to problems, because programmer will forget to free the memory.

b. In Java, memory de-allocation is done automatically through **Garbage Collection**.

iii. Error Handling

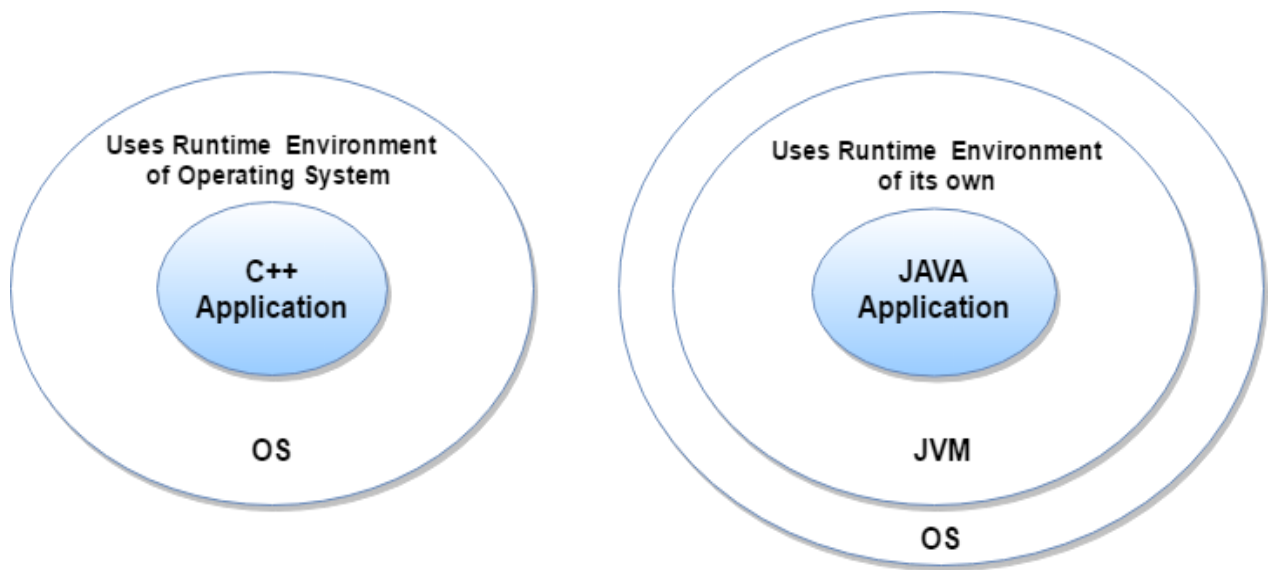
a. Java provides object-oriented exception handling to handle the errors in java program.

b. Using exception handling, all run time errors should be managed by the program.

11. Secure: Java is secured because:

- i. No explicit pointers.

- ii. Java Programs run inside virtual machine environment.



III. Compiling and running a Java program.

A First Simple Program-

```
class Example
{
public static void main(String args[])
{
System.out.println("Welcome to Programming in Java");
}
}
```

1. Open the notepad and type the above program
2. Save the above program with **.java** extension, here file name and class name should be same,
ex: Example.java
3. Open the command prompt and **Compile** the above program
javac Example.java
From the above compilation the java compiler produces a bytecode(.class file)
4. Finally run the program through the
interpreter java Example.java

Output of the program:

Welcome to Programming in Java

Note:

- In Java all code must reside inside a class and name of that class should match the name of the file that holds the program.
- Java is case-sensitive

Compiling the program

- To compile the program, execute the compiler `—javac`, specifying the name of the source file on the command line as shown below

C:\> javac Example.java

- The `—javac`: compiler creates a file called `—Example.class` that contains the bytecode version of the program.
- To run the program, we must use the java interpreter called `—java`. To do so we pass the class name `—Example` as a command-line argument as shown below

C:\> java Example

- When you run the program we get the output:
Welcome to Programming in Java

Description:

- (1) **Class declaration:** `—class Example` declares a class, which is an object- oriented construct. `Sampleone` is a Java identifier that specifies the name of the class to be defined.
- (2) **Opening braces:** Every class definition of Java starts with opening braces and ends with matching one.
- (3) **The main line:** the line `— public static void main(String args[])` — defines a method name `main`. Java application program must include this `main`. This is the starting point of the interpreter from where it starts executing. A Java program can have any number of classes but only one class will have the `main` method.
- (4) **Public:** This key word is an access specifier that declares the `main` method as unprotected and therefore making it accessible to the all other classes.
- (5) **Static:** Static keyword defines the method as one that belongs to the entire class and not for a particular object of the class. The `main` must always be declared as static.
- (6) **Void:** the type modifier `void` specifies that the method `main` does not return any value.
- (7) **The println:** It is a method of the object out of `System` class. It is similar to the `printf` or `cout` of `c` or `c++`. This always appends a newline character to the end of the string i.e, any subsequent output will start on a new line.

A Second Short Program

```
/* This is a short example  
Name of file : Example2.java */  
class Example2{  
public static void main(String args[])  
{  
int n=3;  
System.out.println("the value of n is" +n); n=n+5;  
System.out.print("the new value is"); System.out.println(n);  
}  
}
```

Output:

the value of n is 3

the new value of n is 8

IV. Data Types, Variables, and Arrays

This chapter examines three of Java's most fundamental elements: data types, variables, and arrays. As with all modern programming languages, Java supports several types of data. You may use these types to declare variables and to create arrays. As you will see, Java's approach to these items is clean, efficient, and cohesive.

Java Is a Strongly Typed Language

It is important to state at the outset that Java is a strongly typed language. Indeed, part of Java's safety and robustness comes from this fact. Let's see what this means. First, every variable has a type, every expression has a type, and every type is strictly defined. Second, all assignments, whether explicit or via parameter passing in method calls, are checked for type compatibility. There are no automatic coercions or conversions of conflicting types as in some languages. The Java compiler checks all expressions and parameters to ensure that the types are compatible. Any type mismatches are errors that must be corrected before the compiler will finish compiling the class.

The Primitive Types

Java defines eight *primitive* types of data: **byte**, **short**, **int**, **long**, **char**, **float**, **double**, and **boolean**. The primitive types are also commonly referred to as *simple* types, and both terms will be used in this book. These can be put in four groups:

- **Integers** This group includes **byte**, **short**, **int**, and **long**, which are for whole-valued signed numbers.
- **Floating-point numbers** This group includes **float** and **double**, which represent numbers with fractional precision.
- **Characters** This group includes **char**, which represents symbols in a character set, like letters and numbers.
- **Boolean** This group includes **boolean**, which is a special type for representing true/false values.

You can use these types as-is, or to construct arrays or your own class types. Thus, they form the basis for all other types of data that you can create.

The primitive types represent single values—not complex objects. Although Java is otherwise completely object-oriented, the primitive types are not. They are analogous to the simple types found in most other non-object-oriented languages. The reason for this is efficiency. Making the primitive types into objects would have degraded performance too much.

The primitive types are defined to have an explicit range and mathematical behavior. Languages such as C and C++ allow the size of an integer to vary based upon the dictates of the execution environment. However, Java is different. Because of Java's portability requirement, all data types have a strictly defined range. For example, an **int** is always 32 bits, regardless of the particular platform. This allows programs to be written that are guaranteed to run *without porting* on any machine architecture. While strictly specifying the size of an integer may cause a small loss of performance in some environments, it is necessary in order to achieve portability.

Let's look at each type of data in turn.

Integers

Java defines four integer types: **byte**, **short**, **int**, and **long**. All of these are signed, positive and negative values. Java does not support unsigned, positive-only integers. Many other computer languages support both signed and unsigned integers. However, Java's designers felt that unsigned integers were unnecessary. Specifically, they felt that the concept of *unsigned* was used mostly to specify the behavior of the *high-order bit*, which defines the *sign* of an integer value. Java manages the meaning of the high-order bit differently, by adding a special “unsigned right shift” operator. Thus, the need for an unsigned integer type was eliminated.

The *width* of an integer type should not be thought of as the amount of storage it consumes, but rather as the *behavior* it defines for variables and expressions of that type. The Java run-time environment is free to use whatever size it wants, as long as the types behave as you declared them. The width and ranges of these integer types vary widely, as shown in this table:

Name	Width	Range
long	64	−9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
int	32	−2,147,483,648 to 2,147,483,647
short	16	−32,768 to 32,767
byte	8	−128 to 127

Let's look at each type of integer.

byte

The smallest integer type is **byte**. This is a signed 8-bit type that has a range from −128 to 127. Variables of type **byte** are especially useful when you're working with a stream of data from a network or file. They are also useful when you're working with raw binary data that may not be directly compatible with Java's other built-in types.

Byte variables are declared by use of the **byte** keyword. For example, the following declares two **byte** variables called **b** and **c**:

```
byte b, c;
```

short

short is a signed 16-bit type. It has a range from −32,768 to 32,767. It is probably the least-used Java type. Here are some examples of **short** variable declarations:

```
short  
s;  
short  
t;
```

int

The most commonly used integer type is **int**. It is a signed 32-bit type that has a range from –2,147,483,648 to 2,147,483,647. In addition to other uses, variables of type **int** are commonly employed to control loops and to index arrays.

Although you might think that using a **byte** or **short** would be more efficient than using an **int** in situations in which the larger range of an **int** is not needed, this may not be the case. The reason is that when **byte** and **short** values are used in an expression, they are *promoted* to **int** when the expression is evaluated.

long

long is a signed 64-bit type and is useful for those occasions where an **int** type is not large enough to hold the desired value. The range of a **long** is quite large.

This makes it useful when big, whole numbers are needed. For example, here is a program that computes the number of miles that light will travel in a specified number of days:

```
// Compute distance light travels using long variables.
class Light {
    public static void main(String args[]) {
        int lightspeed;
        long days;
        long seconds;
        long distance;

        // approximate speed of light in miles per second
        lightspeed = 186000;

        days = 1000; // specify number of days here

        seconds = days * 24 * 60 * 60; // convert to seconds

        distance = lightspeed * seconds; // compute distance

        System.out.print("In " + days);
        System.out.print(" days light will travel about ");
        System.out.println(distance + " miles.");
    }
}
```

This program generates the following output:

```
In 1000 days light will travel about 16070400000000 miles.
```

Clearly, the result could not have been held in an **int** variable.

Floating-Point Types

Floating-point numbers, also known as *real* numbers, are used when evaluating expressions that require fractional precision. For example, calculations such as square root, or transcendental functions such as sine and cosine, result in a value whose precision requires a floating-point type. Java implements the standard (IEEE–754) set of floating-point types and operators. There are two kinds of floating-point types, **float** and **double**, which represent single- and double-precision numbers, respectively. Their width and ranges are shown here:

Name	Width in Bits	Approximate Range
double	64	4.9e–324 to 1.8e+308
float	32	1.4e–045 to 3.4e+038

Each of these floating-point types is examined next.

float

The type **float** specifies a *single-precision* value that uses 32 bits of storage. Single precision is faster on some processors and takes half as much space as double precision, but will become imprecise when the values are either very large or very small. Variables of type **float** are useful when you need a fractional component, but don't require a large degree of precision. For example, **float** can be useful when representing dollars and cents.

Here are some example **float** variable declarations:

```
float hightemp, lowtemp;
```

double

Double precision, as denoted by the **double** keyword, uses 64 bits to store a value. Double precision is actually faster than single precision on some modern processors that have been optimized for high-speed mathematical calculations. All transcendental math functions, such as **sin()**, **cos()**, and **sqrt()**, return **double** values. When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, **double** is the best choice.

Here is a short program that uses **double** variables to compute the area of a circle:

```
// Compute the area of a circle.
class Area {
    public static void main(String args[]) {
        double pi, r, a;

        r = 10.8; // radius of circle
        pi = 3.1416; // pi, approximately
        a = pi * r * r; // compute area

        System.out.println("Area of circle is " + a);
    }
}
```

Characters

In Java, the data type used to store characters is **char**. A key point to understand is that Java uses *Unicode* to represent characters. Unicode defines a fully international character set that can represent all of the characters found in all human languages. It is a unification of dozens of character sets, such as Latin, Greek, Arabic, Cyrillic, Hebrew, Katakana, Hangul, and many more. At the time of Java's creation, Unicode required 16 bits. Thus, in Java **char** is a 16-bit type. The range of a **char** is 0 to 65,536. There are no negative **chars**. The standard set of characters known as ASCII still ranges from 0 to 127 as always, and the extended 8-bit character set, ISO-Latin-1, ranges from 0 to 255. Since Java is designed to allow programs to be written for worldwide use, it makes sense that it would use Unicode to represent characters. Of course, the use of Unicode is somewhat inefficient for languages such as English, German, Spanish, or French, whose characters can easily be contained within 8 bits. But such is the price that must be paid for global portability.

Here is a program that demonstrates **char** variables:

```
// Demonstrate char data type.
class CharDemo {
    public static void main(String args[]) {
        char ch1, ch2;

        ch1 = 88; // code for X
        ch2 = 'Y';

        System.out.print("ch1 and ch2: ");
        System.out.println(ch1 + " " + ch2);
    }
}
```

This program displays the following output:

```
ch1 and ch2: X Y
```

Notice that **ch1** is assigned the value 88, which is the ASCII (and Unicode) value that corresponds to the letter X. As mentioned, the ASCII character set occupies the first 127 values in the Unicode character set. For this reason, all the “old tricks” that you may have used with characters in other languages will work in Java, too.

Although **char** is designed to hold Unicode characters, it can also be used as an integer type on which you can perform arithmetic operations. For example, you can add two characters together, or increment the value of a character variable. Consider the following program:


```
// char variables behave like integers.
class CharDemo2 {
    public static void main(String args[]) {
        char ch1;

        ch1 = 'X';
        System.out.println("ch1 contains " + ch1);

        ch1++; // increment ch1
        System.out.println("ch1 is now " + ch1);
    }
}
```

The output generated by this program is shown here:

```
ch1 contains X
ch1 is now Y
```

In the program, **ch1** is first given the value X. Next, **ch1** is incremented. This results in **ch1** containing Y, the next character in the ASCII (and Unicode) sequence.

Booleans

Java has a primitive type, called **boolean**, for logical values. It can have only one of two possible values, **true** or **false**. This is the type returned by all relational operators, as in the case of **a < b**. **boolean** is also the type *required* by the conditional expressions that govern the control statements such as **if** and **for**.

Here is a program that demonstrates the **boolean** type: The output generated by this program is shown here:

```
// Demonstrate boolean values.
class BoolTest {
    public static void main(String args[]) {
        boolean b;

        b = false;
        System.out.println("b is " + b);
        b = true;
        System.out.println("b is " + b);

        // a boolean value can control the if statement
        if(b) System.out.println("This is executed.");

        b = false;
        if(b) System.out.println("This is not executed.");

        // outcome of a relational operator is a boolean value
        System.out.println("10 > 9 is " + (10 > 9));
    }
}
```

```
b is false
b is true
This is executed.
10 > 9 is true
```

There are three interesting things to notice about this program. First, as you can see, when a **boolean** value is output by **println()**, "true" or "false" is displayed. Second, the value of a **boolean** variable is sufficient, by itself, to control the **if** statement. There is no need to write an **if** statement like this:

```
if(b == true) ...
```

Third, the outcome of a relational operator, such as **<**, is a **boolean** value. This is why the expression **10>9** displays the value "true." Further, the extra set of parentheses around **10>9** is necessary because the **+** operator has a higher precedence than the **>**.

Variables

The variable is the basic unit of storage in a Java program. A variable is defined by the combination of an identifier, a type, and an optional initializer. In addition, all variables have a scope, which defines their visibility, and a lifetime. These elements are examined next.

Declaring a Variable

In Java, all variables must be declared before they can be used. The basic form of a variable declaration is shown here:

```
type identifier [ = value ][, identifier [= value ] ...];
```

Here, *type* is one of Java's atomic types, or the name of a class or interface. (Class and interface types are discussed later in [Part I](#) of this book.) The *identifier* is the name of the variable. You can initialize the variable by specifying an equal sign and a value. Keep in mind that the initialization expression must result in a value of the same (or compatible) type as that specified for the variable. To declare more than one variable of the specified type, use a comma-separated list.

Here are several examples of variable declarations of various types. Note that some include an initialization.

```
int a, b, c;           // declares three ints, a, b, and c.
int d = 3, e, f = 5;   // declares three more ints, initializing
                        // d and f.
byte z = 22;           // initializes z.
double pi = 3.14159;   // declares an approximation of pi.
char x = 'x';          // the variable x has the value 'x'.
```

The identifiers that you choose have nothing intrinsic in their names that indicates their type. Java allows any properly formed identifier to have any declared type.

Dynamic Initialization

Although the preceding examples have used only constants as initializers, Java allows variables to be initialized dynamically, using any expression valid at the time the variable is declared.

For example, here is a short program that computes the length of the hypotenuse of a right triangle given the lengths of its two opposing sides:

```
// Demonstrate dynamic initialization.
class DynInit {
    public static void main(String args[]) {
        double a = 3.0, b = 4.0;

        // c is dynamically initialized
        double c = Math.sqrt(a * a + b * b);

        System.out.println("Hypotenuse is " + c);
    }
}
```

Here, three local variables—**a**, **b**, and **c**—are declared. The first two, **a** and **b**, are initialized by constants. However, **c** is initialized dynamically to the length of the hypotenuse (using the Pythagorean theorem). The program uses another of Java's built-in methods, **sqrt()**, which is a member of the **Math** class, to compute the square root of its argument. The key point here is that the initialization expression may use any element valid at the time of the initialization, including calls to methods, other variables, or literals.

The Scope and Lifetime of Variables

So far, all of the variables used have been declared at the start of the **main()** method. However, Java allows variables to be declared within any block. As explained in [Chapter 2](#), a block is begun with an opening curly brace and ended by a closing curly brace. A block defines a *scope*. Thus, each time you start a new block, you are creating a new scope. A scope determines what objects are visible to other parts of your program. It also determines the lifetime of those objects.

It is not uncommon to think in terms of two general categories of scopes: global and local. However, these traditional scopes do not fit well with Java's strict, object-oriented model. While it is possible to create what amounts to being a global scope, it is by far the exception, not the rule. In Java, the two major scopes are those defined by a class and those defined by a method. Even this distinction is somewhat artificial. However, since the class scope has several unique properties and attributes that do not apply to the scope defined by a method, this distinction makes some sense. Because of the differences, a discussion of class scope (and variables declared within it) is deferred until [Chapter 6](#), when classes are described. For now, we will only examine the scopes defined by or within a method.

The scope defined by a method begins with its opening curly brace. However, if that method has parameters, they too are included within the method's scope. A method's scope ends with its closing curly brace. This block of code is called the *method body*.

As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope. Thus, when you declare a variable within a scope, you are localizing that variable and protecting it from unauthorized access and/or modification. Indeed, the scope rules provide the foundation for encapsulation. A variable declared within a block is called a *local variable*.

Scopes can be nested. For example, each time you create a block of code, you are creating a new, nested scope. When this occurs, the outer scope encloses the inner scope. This means that objects declared in the outer scope will be visible to

code within the inner scope. However, the reverse is not true. Objects declared within the inner scope will not be visible outside it.

To understand the effect of nested scopes, consider the following program:

```
// Demonstrate block scope.
class Scope {
    public static void main(String args[]) {
        int x; // known to all code within main

        x = 10;
        if(x == 10) { // start new scope
            int y = 20; // known only to this block

            // x and y both known here.
            System.out.println("x and y: " + x + " " + y);
            x = y * 2;
        }
        // y = 100; // Error! y not known here

        // x is still known here.
        System.out.println("x is " + x);
    }
}
```

As the comments indicate, the variable **x** is declared at the start of **main()**'s scope and is accessible to all subsequent code within **main()**. Within the **if** block, **y** is declared. Since a block defines a scope, **y** is only visible to other code within its block. This is why outside of its block, the line **y = 100;** is commented out. If you remove the leading comment symbol, a compile-time error will occur, because **y** is not visible outside of its block. Within the **if** block, **x** can be used because code within a block (that is, a nested scope) has access to variables declared by an enclosing scope.

Within a block, variables can be declared at any point, but are valid only after they are declared. Thus, if you define a variable at the start of a method, it is available to all of the code within that method. Conversely, if you declare a variable at the end of a block, it is effectively useless, because no code will have access to it. For example, this fragment is invalid because **count** cannot be used prior to its declaration:

```
// This fragment is wrong!
count = 100; // oops! cannot use count before it is
declared! int count;
```

Here is another important point to remember: variables are created when their scope is entered, and destroyed when their scope is left. This means that a variable will not hold its value once it has gone out of scope. Therefore, variables declared within a method will not hold their values between calls to that method. Also, a variable declared within a block will lose its value when the block is left. Thus, the lifetime of a variable is confined to its scope.

If a variable declaration includes an initializer, then that variable will be reinitialized each time the block in which it is declared is entered. For example, consider the next program:

```
// Demonstrate lifetime of a variable.
class LifeTime {
    public static void main(String args[]) {
        int x;

        for(x = 0; x < 3; x++) {
            int y = -1; // y is initialized each time block is entered
            System.out.println("y is: " + y); // this always prints -1
            y = 100;
            System.out.println("y is now: " + y);
        }
    }
}
```

The output generated by this program is shown here:

```
y is: -1
y is now: 100
y is: -1
y is now: 100
y is: -1
y is now: 100
```

As you can see, **y** is reinitialized to -1 each time the inner **for** loop is entered. Even though it is subsequently assigned the value 100, this value is lost.

One last point: Although blocks can be nested, you cannot declare a variable to have the same name as one in an outer scope. For example, the following program is illegal:

```
// This program will not compile
class ScopeErr {
    public static void main(String args[]) {
        int bar = 1;
        {
            // creates a new scope
            int bar = 2; // Compile-time error - bar already defined!
        }
    }
}
```

Type Conversion and Casting

If you have previous programming experience, then you already know that it is fairly common to assign a value of one type to a variable of another type. If the two types are compatible, then Java will perform the conversion automatically. For example, it is always possible to assign an **int** value to a **long** variable.

However, not all types are compatible, and thus, not all type conversions are implicitly allowed. For instance, there is no automatic conversion defined from **double** to **byte**. Fortunately, it is still possible to obtain a conversion between incompatible types. To do so, you must use a *cast*, which performs an explicit conversion between incompatible types. Let's look at both automatic type conversions and casting.

Java's Automatic Conversions

When one type of data is assigned to another type of variable, an *automatic type conversion* will take place if the following two conditions are met:

- The two types are compatible.
- The destination type is larger than the source type.

When these two conditions are met, a *widening conversion* takes place. For example, the **int** type is always large enough to hold all valid **byte** values, so no explicit cast statement is required.

For widening conversions, the numeric types, including integer and floating-point types, are compatible with each other. However, there are no automatic conversions from the numeric types to **char** or **boolean**. Also, **char** and **Boolean** are not compatible with each other.

As mentioned earlier, Java also performs an automatic type conversion when storing a literal integer constant into variables of type **byte**, **short**, **long**, or **char**.

Casting Incompatible Types

Although the automatic type conversions are helpful, they will not fulfill all needs. For example, what if you want to assign an **int** value to a **byte** variable? This conversion will not be performed automatically, because a **byte** is smaller than an **int**. This kind of conversion is sometimes called a *narrowing conversion*, since you are explicitly making the value narrower so that it will fit into the target type.

To create a conversion between two incompatible types, you must use a cast.

A *cast* is simply an explicit type conversion. It has this general form: *(target-type) value*

Here, *target-type* specifies the desired type to convert the specified value to. For example, the following fragment casts an **int** to a **byte**. If the integer's value is larger than the range of a **byte**, it will be reduced modulo (the remainder of an integer division by the) **byte**'s range.

```
int a; byte b;  
// ...  
b = (byte) a;
```

A different type of conversion will occur when a floating-point value is assigned to an integer type: *truncation*. As you know, integers do not have fractional components. Thus, when a floating-point value is assigned to an integer type, the fractional component is lost. For example, if the value 1.23 is assigned to an integer, the resulting value will simply be 1. The 0.23 will have been

truncated. Of course, if the size of the whole number component is too large to fit into the target integer type, then that value will be reduced modulo the target type's range.

The following program demonstrates some type conversions that require casts:

```
// Demonstrate casts.
class Conversion {
    public static void main(String args[]) {
        byte b;
        int i = 257;
        double d = 323.142;

        System.out.println("\nConversion of int to byte.");
        b = (byte) i;
        System.out.println("i and b " + i + " " + b);

        System.out.println("\nConversion of double to int.");
        i = (int) d;
        System.out.println("d and i " + d + " " + i);

        System.out.println("\nConversion of double to byte.");
        b = (byte) d;
        System.out.println("d and b " + d + " " + b);
    }
}
```

This program generates the following output:

Conversion of int to byte. i and b 257 1

Conversion of double to int. d and i 323.142 323

Conversion of double to byte. d and b 323.142 67

Let's look at each conversion. When the value 257 is cast into a **byte** variable, the result is the remainder of the division of 257 by 256 (the range of a **byte**), which is 1 in this case. When the **d** is converted to an **int**, its fractional component is lost. When **d** is converted to a **byte**, its fractional component is lost, *and* the value is reduced modulo 256, which in this case is 67.

Automatic Type Promotion in Expressions

In addition to assignments, there is another place where certain type conversions may occur: in expressions. To see why, consider the following. In an expression, the precision required of an intermediate value will sometimes exceed the range of either operand. For example, examine the following expression:

```
byte a = 40; byte b = 50; byte c = 100;  
int d = a * b / c;
```

The result of the intermediate term **a * b** easily exceeds the range of either of its **byte** operands. To handle this kind of problem, Java automatically promotes each **byte**, **short**, or **char** operand to **int** when evaluating an expression. This means that the subexpression **a*b** is performed using integers—not bytes. Thus, 2,000, the result of the intermediate expression, **50 * 40**, is legal even though **a** and **b** are both specified as type **byte**.

As useful as the automatic promotions are, they can cause confusing compile-time errors. For example, this seemingly correct code causes a problem:

```
byte b = 50;  
b = b * 2; // Error! Cannot assign an int to a byte!
```

The code is attempting to store **50 * 2**, a perfectly valid **byte** value, back into a **byte** variable. However, because the operands were automatically promoted to **int** when the expression was evaluated, the result has also been promoted to **int**. Thus, the result of the expression is now of type **int**, which cannot be assigned to a **byte** without the use of a cast. This is true even if, as in this particular case, the value being assigned would still fit in the target type.

In cases where you understand the consequences of overflow, you should use an explicit cast, such as

```
byte b = 50;  
b = (byte) (b * 2);
```

which yields the correct value of 100.

The Type Promotion Rules

Java defines several *type promotion* rules that apply to expressions. They are as follows: First, all **byte**, **short**, and **char** values are promoted to **int**, as just described. Then, if one operand is a **long**, the whole expression is promoted to

long. If one operand is a **float**, the entire expression is promoted to **float**. If any of the operands are **double**, the result is **double**.

The following program demonstrates how each value in the expression gets promoted to match the second argument to each binary operator:

```
class Promote {
    public static void main(String args[]) {
        byte b = 42;
        char c = 'a';
        short s = 1024;
        int i = 50000;
        float f = 5.67f;
        double d = .1234;
        double result = (f * b) + (i / c) - (d * s);
        System.out.println((f * b) + " + " + (i / c) + " - " + (d * s));
        System.out.println("result = " + result);
    }
}
```

Let's look closely at the type promotions that occur in this line from the program:

```
double result = (f * b) + (i / c) - (d * s);
```

In the first subexpression, **f * b**, **b** is promoted to a **float** and the result of the subexpression is **float**. Next, in the subexpression **i/c**, **c** is promoted to **int**, and the result is of type **int**. Then, in **d * s**, the value of **s** is promoted to **double**, and the type of the subexpression is **double**. Finally, these three intermediate values, **float**, **int**, and **double**, are considered. The outcome of **float** plus an **int** is a **float**. Then the resultant **float** minus the last **double** is promoted to **double**, which is the type for the final result of the expression.

Arrays

An *array* is a group of like-typed variables that are referred to by a common name. Arrays of any type can be created and may have one or more dimensions. A specific element in an array is accessed by its index. Arrays offer a convenient means of grouping related information.

One-Dimensional Arrays

A *one-dimensional array* is, essentially, a list of like-typed variables. To create an array, you first must create an array variable of the desired type. The general form of a one-dimensional array declaration is

```
type var-name[ ];
```

Here, *type* declares the element type (also called the base type) of the array. The element type determines the data type of each element that comprises the array. Thus, the element type for the array determines what type of data the array will hold. For example, the following declares an array named **month_days** with the type “array of int”:

```
int month_days[];
```

Although this declaration establishes the fact that **month_days** is an array variable, no array actually exists. To link **month_days** with an actual, physical array of integers, you must allocate one using **new** and assign it to **month_days**. **new** is a special operator that allocates memory.

You will look more closely at **new** in a later chapter, but you need to use it now to allocate memory for arrays. The general form of **new** as it applies to one-dimensional arrays appears as follows:

```
array-var = new type [size];
```

Here, *type* specifies the type of data being allocated, *size* specifies the number of elements in the array, and *array-var* is the array variable that is linked to the array. That is, to use **new** to allocate an array, you must specify the type and number of elements to allocate. The elements in the array allocated by **new** will automatically be initialized to zero (for numeric types), **false** (for **boolean**), or **null** (for reference types, which are described in a later chapter). This example allocates a 12-element array of integers and links them to **month_days**:

```
month_days = new int[12];
```

After this statement executes, **month_days** will refer to an array of 12 integers. Further, all elements in the array will be initialized to zero.

Let's review: Obtaining an array is a two-step process. First, you must declare a variable of the desired array type. Second, you must allocate the memory that will hold the array, using **new**, and assign it to the array variable. Thus, in Java all arrays are dynamically allocated. If the concept of dynamic allocation is unfamiliar to you, don't worry. It will be described at length later in this book.

Once you have allocated an array, you can access a specific element in the array by specifying its index within square brackets. All array indexes start at zero. For example, this statement assigns the value 28 to the second element of **month_days**:

```
month_days[1] = 28;
```

The next line displays the value stored at index 3:

```
System.out.println(month_days[3]);
```

Putting together all the pieces, here is a program that creates an array of the number of days in each month:

```
// Demonstrate a one-dimensional array.
class Array {
    public static void main(String args[]) {
        int month_days[];
        month_days = new int[12];
        month_days[0] = 31;
        month_days[1] = 28;
        month_days[2] = 31;
        month_days[3] = 30;
        month_days[4] = 31;
        month_days[5] = 30;
        month_days[6] = 31;
        month_days[7] = 31;
        month_days[8] = 30;
        month_days[9] = 31;
        month_days[10] = 30;
        month_days[11] = 31;
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

When you run this program, it prints the number of days in April. As mentioned, Java array indexes start with zero, so the number of days in April is **month_days[3]** or 30.

It is possible to combine the declaration of the array variable with the allocation of the array itself, as shown here:

```
int month_days[] = new int[12];
```

This is the way that you will normally see it done in professionally written Java programs.

Arrays can be initialized when they are declared. The process is much the same as that used to initialize the simple types. An *array initializer* is a list of comma-separated expressions surrounded by curly braces. The commas separate the values of the array elements. The array will automatically be created large enough to hold the number of elements you specify in the array initializer. There is no need to use **new**. For example, to store the number of days in each month, the following code creates an initialized array of integers:

```
// An improved version of the previous program.
class AutoArray {
    public static void main(String args[]) {

        int month_days[] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31,
                               30, 31 };
        System.out.println("April has " + month_days[3] + " days.");
    }
}
```

When you run this program, you see the same output as that generated by the previous version.

Java strictly checks to make sure you do not accidentally try to store or reference values outside of the range of the array. The Java run-time system will check that all array indexes are in the correct range. For example, the run-time system will check the value of each index into **month_days** to make sure that it is between 0 and 11 inclusive. If you try to access elements outside the range of the array (negative numbers or numbers greater than the length of the array), you will cause a run-time error.

Here is one more example that uses a one-dimensional array. It finds the average of a set of numbers.

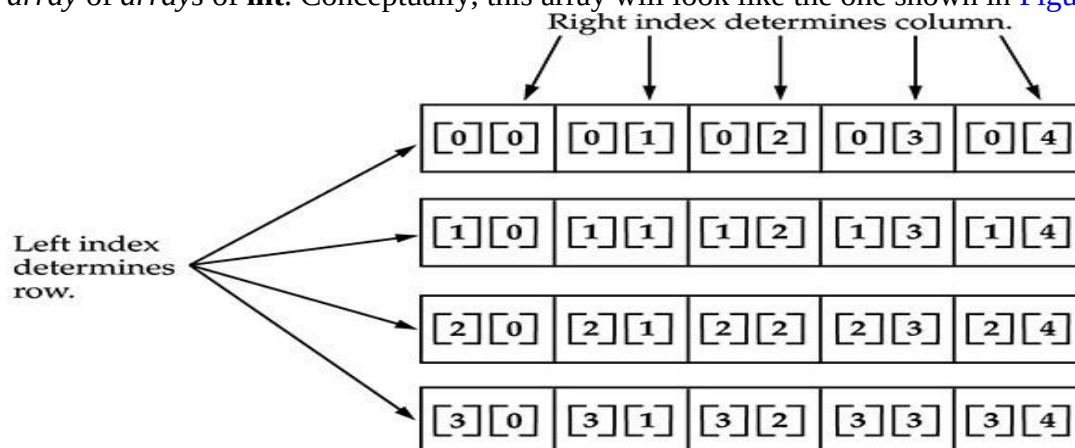
```
// Average an array of values.
class Average {
    public static void main(String args[]) {
        double nums[] = {10.1, 11.2, 12.3, 13.4, 14.5};
        double result = 0;
        int i;
        for(i=0; i<5; i++)
            result = result + nums[i];
        System.out.println("Average is " + result / 5);
    }
}
```

Multidimensional Arrays

In Java, *multidimensional arrays* are implemented as arrays of arrays. To declare a multidimensional array variable, specify each additional index using another set of square brackets. For example, the following declares a two-dimensional array variable called **twoD**:

```
int twoD[][] = new int[4][5];
```

This allocates a 4 by 5 array and assigns it to **twoD**. Internally, this matrix is implemented as an *array of arrays of int*. Conceptually, this array will look like the one shown in [Figure 3-1](#).



Given: `int twoD [] [] = new int [4] [5] ;`

Figure 3-1 A conceptual view of a 4 by 5, two-dimensional array

The following program numbers each element in the array from left to right, top to bottom, and then displays these values:

```
// Demonstrate a two-dimensional array.
class TwoDArray {
    public static void main(String args[]) {
        int twoD[][] = new int[4][5];
        int i, j, k = 0;

        for(i=0; i<4; i++)
            for(j=0; j<5; j++) {
                twoD[i][j] = k;
                k++;
            }

        for(i=0; i<4; i++) {
            for(j=0; j<5; j++)
                System.out.print(twoD[i][j] + " ");
            System.out.println();
        }
    }
}
```

This program generates the following output:

```
0 1 2 3 4
5 6 7 8 9
10 11 12 13 14
15 16 17 18 19
```

When you allocate memory for a multidimensional array, you need only specify the memory for the first (leftmost) dimension. You can allocate the remaining dimensions separately. For example, this following code allocates memory for the first dimension of **twoD** when it is declared. It allocates the second dimension separately.

```
int twoD[][] = new int[4][]; twoD[0] = new int[5]; twoD[1] = new
int[5]; twoD[2] = new int[5]; twoD[3] = new int[5];
```

While there is no advantage to individually allocating the second dimension arrays in this situation, there may be in others. For example, when you allocate dimensions individually, you do not need to allocate the same number of elements for each dimension. As stated earlier, since multidimensional arrays are actually arrays of arrays, the length of each array is under your control. For example, the following program creates a two-dimensional array in which the sizes of the second dimension are unequal:

```
// Manually allocate differing size second dimensions.
class TwoDAgain {
    public static void main(String args[]) {
        int twoD[][] = new int[4][];
        twoD[0] = new int[1];
        twoD[1] = new int[2];
        twoD[2] = new int[3];
        twoD[3] = new int[4];

        int i, j, k = 0;

        for(i=0; i<4; i++)
            for(j=0; j<i+1; j++) {
                twoD[i][j] = k;
                k++;
            }

        for(i=0; i<4; i++) {
            for(j=0; j<i+1; j++)
                System.out.print(twoD[i][j] + " ");
            System.out.println();
        }
    }
}
```

This program generates the following output:

```
0
1 2
3 4 5
6 7 8 9
```

The array created by this program looks like this:

[0][0]			
[1][0]	[1][1]		
[2][0]	[2][1]	[2][2]	
[3][0]	[3][1]	[3][2]	[3][3]

The use of uneven (or irregular) multidimensional arrays may not be appropriate for many applications, because it runs contrary to what people expect to find when a multidimensional array is encountered. However, irregular arrays can be used effectively in some situations. For example, if you need a very large two-dimensional array that is sparsely populated (that is, one in which not all of the elements will be used), then an irregular array might be a perfect solution.

It is possible to initialize multidimensional arrays. To do so, simply enclose each dimension's initializer within its own set of curly braces. The following program creates a matrix where each element contains the product of the row and column indexes. Also notice that you can use expressions as well as literal values inside of array initializers.

```
// Initialize a two-dimensional array.
class Matrix {
    public static void main(String args[]) {
        double m[][] = {
            { 0*0, 1*0, 2*0, 3*0 },
            { 0*1, 1*1, 2*1, 3*1 },
            { 0*2, 1*2, 2*2, 3*2 },
            { 0*3, 1*3, 2*3, 3*3 }
        };
        int i, j;

        for(i=0; i<4; i++) {
            for(j=0; j<4; j++)
                System.out.print(m[i][j] + " ");
            System.out.println();
        }
    }
}
```

When you run this program, you will get the following output:

```
0.0 0.0 0.0 0.0
0.0 1.0 2.0 3.0
0.0 2.0 4.0 6.0
0.0 3.0 6.0 9.0
```

As you can see, each row in the array is initialized as specified in the initialization lists.

Let's look at one more example that uses a multidimensional array. The following program creates a 3 by 4 by 5, three-dimensional array. It then loads each element with the product of its indexes. Finally, it displays these products.


```
// Demonstrate a three-dimensional array.
class ThreeDMatrix {
    public static void main(String args[]) {
        int threeD[][][] = new int[3][4][5];
        int i, j, k;

        for(i=0; i<3; i++)
            for(j=0; j<4; j++)
                for(k=0; k<5; k++)
                    threeD[i][j][k] = i * j * k;

        for(i=0; i<3; i++) {
            for(j=0; j<4; j++) {
                for(k=0; k<5; k++)
                    System.out.print(threeD[i][j][k] + " ");
                System.out.println();
            }
            System.out.println();
        }
    }
}
```

This program generates the following output:

```
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0

0 0 0 0 0
0 1 2 3 4
0 2 4 6 8
0 3 6 9 12

0 0 0 0 0
0 2 4 6 8
0 4 8 12 16
0 6 12 18 24
```

Alternative Array Declaration Syntax

There is a second form that may be used to declare an array:

type[] var-name;

Here, the square brackets follow the type specifier, and not the name of the array variable. For example, the following two declarations are equivalent:

```
int a1[] = new int[3]; int[] a2 = new int[3];
```

The following declarations are also equivalent:

```
char twod1[][] = new char[3][4];  
char[][] twod2 = new char[3][4];
```

This alternative declaration form offers convenience when declaring several arrays at the same time. For example,

```
int[] nums, nums2, nums3; // create three arrays
```

creates three array variables of type **int**. It is the same as writing

```
int nums[], nums2[], nums3[]; // create three arrays
```

The alternative declaration form is also useful when specifying an array as a return type for a method. Both forms are used in this book.

Operators

Java's operators can be divided into **following** groups:

1. Arithmetic operators
2. The Bitwise operators
3. Relational operators
4. Boolean Logical operators
5. The assignment operator
6. The ? (**Ternary**) operator

1. Arithmetic Operators: Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra.

<u>Operator</u>	<u>Result</u>
+	Addition (unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
– =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

- ☐ The operands of the arithmetic operators must be of a **numeric type**.
- ☐ We cannot use them on **boolean** types, but we can use them on **char** types, since the **char** type in Java is, essentially, a subset of **int**.
- ☐ The **unary minus** operator negates its single operand.
- ☐ The **unary plus** operator just returns the operand value.
- ☐ When the **division operator** is applied to an integer type, there will be no fractional component attached to the result.

// Demonstrate the basic arithmetic operators.

```
class BasicMath {
    public static void main(String args[]) {
        // arithmetic using integers
        System.out.println("Integer Arithmetic");
        int a = 1 + 1;
        int b = a * 3;
        int c = b / 4;
        int d = c - a;
        int e = -d;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
        System.out.println("d = " + d);
        System.out.println("e = " + e);

        // arithmetic using doubles
        System.out.println("\nFloating Point Arithmetic");
        double da = 1 + 1;
        double db = da * 3;
        double dc = db / 4;
        double dd = dc - a;
        double de = -dd;
        System.out.println("da = " + da);
        System.out.println("db = " + db);
        System.out.println("dc = " + dc);
        System.out.println("dd = " + dd);
        System.out.println("de = " + de);
    }
}
```

Output:

Integer Arithmetic

a = 2

b = 6

c = 1

d = -1

e = 1

Floating Point Arithmetic

da = 2.0

db = 6.0

dc = 1.5

dd = -0.5

de = 0.5

The Modulus Operator (%) : The modulus operator, %, returns the remainder of a division operation. It can be applied to floating-point types as well as integer types.

// Demonstrate the % operator.

```
class Modulus {
    public static void main(String args[]) {
        int x = 42;
        double y = 42.25;
        System.out.println("x mod 10 = " + x % 10);
        System.out.println("y mod 10 = " + y % 10);
    }
}
```

Output:

x mod 10 = 2

y mod 10 = 2.25

Arithmetic Compound Assignment Operators (+=, -=, *=, /=, %=)

Any statement of the form **var = var op expression;** can be written as **var op= expression;**

// Demonstrate several assignment operators.

```
class OpEquals {
    public static void main(String args[]) {
        int a = 1;
        int b = 2;
        int c = 3;
        a += 5;
        b *= 4;
        c += a * b;
        c %= 6;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    }
}
```

Output:

```
a = 6
b = 8
c = 3
```

Benefits of compound assignment operators

1. They save a bit of typing.
2. In some cases they are more efficient than are their equivalent long forms.

Increment and Decrement (++ and --)

The increment operator (++)

The increment operator increases its operand by one.

$x = x + 1$; can be rewritten like this by use of the increment operator: $x++$;

The decrement operator (--)

The decrement operator decreases its operand by one.

$x = x - 1$; can be rewritten like this by use of the increment operator: $x--$;

// Demonstrate ++.

```
class IncDec {
    public static void main(String args[]) {
        int a = 1;
        int b = 2;
        int c;
        int d;
        c = ++b;
        d = a++;
        c++;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
        System.out.println("d = " + d);
    }
}
```

Output:

```
a = 2
b = 3
c = 4
d = 1
```

- ☐ These operators can appear both in **prefix** or **postfix** form.

Prefix form: The operand is incremented or decremented before the value is obtained for use in the expression.

$x = 42$; equal to $x = x + 1$;
 $y = ++x$; equal to $y = x$;

Y = 43, X = 43

Postfix form: The previous value is obtained for use in the expression, and then the operand is modified.

$x = 42$; equal to
 $y = x++$;

Y=x;
X = x + 1;

Y = 42 , X = 43

2. The Bitwise Operators

- Java defines several bitwise operators that can be applied to the **integer types: long, int, short, char, and byte.**
- These operators act upon the **individual bits of their operands.**

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

2's complement

- 1) Java uses an **encoding** known as **two's complement** to represent negative integers.
 - i. Write the binary number for +ve value
 - ii. Invert the bits (changing 1's to 0's and vice versa)
 - iii. Add 1 to the result obtained in step ii.

Example:

–42 is represented by inverting all of the bits in 42, or 00101010, which yields 11010101, then adding 1, which results in 11010110, or –42.

2) Decoding

To decode a negative number, first invert all of the bits, then add 1.

For example, -42, or 11010110 inverted, yields 00101001, or 41, so when you add 1, you get 42.

- 3) The leftmost bit (high-order bit) determines sign of an integer. If the bit is 1 then it is a negative number. If the bit is 0 then it is a positive number.

The Bitwise Logical Operators

The bitwise logical operators are &, |, ^, and ~.

The Bitwise NOT

Also called the **bitwise complement**, the **unary NOT operator**, ~, inverts all of the bits of its operand.

The Bitwise AND

The AND operator, &, produces a **1** bit if both operands are also **1**.

A **zero** is produced in all other cases.

The Bitwise OR

The OR operator, |, combines bits such that if either of the bits in the operands is a 1, then the resultant bit is a 1.

The Bitwise XOR

The XOR operator, ^, combines bits such that if exactly one operand is 1, then the result is 1. Otherwise, the result is **zero**.

A	B	A B	A & B	A ^ B	~A
0	0	0	0	0	1
1	0	1	0	1	0
0	1	1	0	1	1
1	1	1	1	0	0

// Demonstrate the bitwise logical operators.

```
class BitLogic
{
    public static void main(String args[]) {
        int a = 3; // 0011 in binary
        int b = 6; // 0110 in binary
```

Output

a|b = 7

a & b = 2

a ^ b = 3

```

        int c = a | b;
        int d = a & b;
        int e = a ^ b;

        System.out.println("    a|b = " + a | b);
        System.out.println("    a&b = " + a & b);
        System.out.println("    a^b = " + a ^ b);
    }
}

```

The Left Shift

The left shift operator, `<<`, shifts all of the bits in a value to the left a specified number of times. It has this general form:

value `<<` num

- a) Here, **num** specifies the number of positions to left-shift the value in value.
- b) For each shift left, the high-order bit is shifted out (and lost), and a zero is brought in on the right.

// Left shifting a byte value.

```

class ByteShift
{
    public static void main(String args[]) {
        byte a = 64, b;
        int i;

        i = a << 2;
        b = (byte) (a << 2);
        System.out.println("Original value of a: " + a);
        System.out.println("i and b: " + i + " " + b);
    }
}

```

Output:

Original value of a: 64

i and b: 256 0

The Right Shift

- 1) The right shift operator, `>>`, shifts all of the bits in a value to the right a specified number of times. It has this general form:

value `>>` num

- 2) Here, **num** specifies the number of positions to right-shift the value in **value**.
- 3) Each time you shift a value to the right, it **divides that value by two—and discards any remainder**.

```

int a = 35;
a = a >> 2; // a contains 8

```



```
00100011 35
  >> 2
00001000 8
```

- 4) When we are shifting right, the top (leftmost) bits exposed by the right shift are filled in with the previous contents of the top bit. This is called **sign extension** and serves to preserve the sign of negative numbers.

```
11111000 -8
  >> 1
11111100 -4
```

The Unsigned Right Shift (>>>)

Java's unsigned, shift right operator, >>>, which always **shifts zeros into the high-order bit**.

```
int a = -1;
a = a >>> 24;

11111111 11111111 11111111 11111111  -1 in binary as an int
  >>>24

00000000 00000000 00000000 11111111 255 in binary as an int
```

Bitwise Operator Compound Assignments

a = a >> 4; can be written as a >>= 4;

```
class OpBit {
    public static void main(String args[]) {
        int a = 1;
        int b = 2;
        int c = 3;
        a |= 4;
        b >>= 1;
        c <<= 1;
        a ^= c;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("c = " + c);
    }
}
```

Output

a=3

b=1

c=6

3. Relational Operators

- 1) The relational operators determine the relationship that one operand has to the other. Specifically, they determine equality and ordering.
- 2) The outcome of these operations is a boolean value.

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

- 3) The relational operators are most frequently used in the expressions that control the **if** statement and the various **loop statements**.

```
int a = 4;
int b = 1;
boolean c = a < b;
```

Output:

false

4. Boolean Logical Operators

The Boolean logical operators shown here operate only on **boolean** operands.

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT

&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

Example

A	B	A B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

// Demonstrate the boolean logical operators.

```

class BoolLogic {
    public static void main(String args[]) {
        boolean a = true;
        boolean b = false;
        boolean c = a | b;
        boolean d = a & b;
        System.out.println( a);
        System.out.println(b);
        System.out.println( c);
        System.out.println(d);
    }
}

```

Output:

```

true
false
true
false

```

Short-Circuit Logical Operators(&& , ||)

The secondary versions of the **Boolean AND (&)** and **OR (|)** operators are known as Short-circuit operators.

If we use short-circuit operators `||` and `&&`, Java will not bother to evaluate the right hand operand when the outcome of the expression can be determined by the left operand alone.

Short-circuit AND - && (Conditional-and)

```
class ShortCktAnd
{
    public static void main(String args[])
    {
        int num = 20;
        int denom = 0;

        if (denom != 0 && num / denom > 10)
        {
            System.out.println("Inside If");
        }
        System.out.println("After if");
    }
}
```

Output:

After if

Short-circuit OR - || (Conditional-or)

```
class ShortCktOr
{
    public static void main(String args[])
    {
        int num = 20;
        int denom = 0;

        if (denom == 0 || num / denom > 10)
        {
            System.out.println("Inside If");
        }
        System.out.println("After if");
    }
}
```

Ouput:

Inside if

After if

5. The Assignment Operator

The assignment operator is the single equal sign, `=`.

It's general form is:

```
var = expression;
```

Here, the type of **var** must be compatible with the type of **expression**.

Example:

```
int x, y, z;
```

x = y = z = 100;

The ternary / three-way operator (?)

The ? has this general form:

expression1 ? expression2 : expression3

- expression1 can be any expression that evaluates to a boolean value.
- If expression1 is true, then expression2 is evaluated; otherwise, expression3 is evaluated.
- Both expression2 and expression3 are required to return the same (or compatible) type, which can't be void.
- It will be used to replace certain types of if-then-else statements.

Example

// Demonstrate ?.

```
class Ternary {
    public static void main(String args[])
    {
        int num = 20;
        int denom = 0;

        int ratio = denom == 0 ? 0 : num / denom;
        System.out.println(ratio);
    }
}
```

Output:

0

Operator Precedence

- 1) In the below table, Operators with higher precedence are evaluated before operators with relatively lower precedence. Operators on the same line have equal precedence.
- 2) When operators of equal precedence appear in the same expression, All binary operators except for the assignment operators are evaluated from left to right; assignment operators are evaluated right to left.

Highest						
postfix ++	postfix --					
prefix ++	prefix --	~	!	unary +	unary -	(type-cast)

*	/	%				
+	-					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
&&						
?:						
->						
=	op=					
Lowest						

Using Parentheses

- 1) **Parentheses** raise the precedence of the operations that are inside them.
- 2) **Parentheses** can sometimes be used to help clarify the meaning of an expression.

Example

$a \gg b + 3$ → This expression first adds 3 to b and then shifts a right by that result.

If we want to first shift a right by b positions and then add 3 to that result, we will need to parenthesize the expression like this:

$(a \gg b) + 3$

VI. Introducing Classes

The class is at the core of Java. It is the logical construct upon which the entire Java language is built because it defines the shape and nature of an object. As such, the class forms the basis for object-oriented programming in Java. Any concept you wish to implement in a Java program must be encapsulated within a class.

Because the class is so fundamental to Java, this and the next few chapters will be devoted to it. Here, you will be introduced to the basic elements of a class and learn how a class can be used to create objects. You will also learn about methods, constructors, and the **this** keyword.

Class Fundamentals

Classes have been used since the beginning of this book. However, until now, only the most rudimentary form of a class has been shown. The classes created in the preceding chapters primarily exist simply to encapsulate the **main()** method, which has been used to demonstrate the basics of the Java syntax. As you will see, classes are substantially more powerful than the limited ones presented so far.

Perhaps the most important thing to understand about a class is that it defines a new data type. Once defined, this new type can be used to create objects of that type. Thus, a class is a *template* for an object, and an object is an *instance* of a class. Because an object is an instance of a class, you will often see the two words *object* and *instance* used interchangeably.

The General Form of a Class

When you define a class, you declare its exact form and nature. You do this by specifying the data that it contains and the code that operates on that data. While very simple classes may contain only code or only data, most real-world classes contain both. As you will see, a class' code defines the interface to its data.

A class is declared by use of the **class** keyword. The classes that have been used up to this point are actually very limited examples of its complete form.

Classes can (and usually do) get much more complex. A simplified general form of a **class** definition is shown here:

```
class classname {  
    type instance-variable1;  
    type instance-variable2;  
    // ...  
    type instance-variableN;  
  
    type methodname1(parameter-list) {  
        // body of method  
    }  
    type methodname2(parameter-list) {  
        // body of method  
    }  
    // ...  
    type methodnameN(parameter-list) {  
        // body of method  
    }  
}
```

The data, or variables, defined within a **class** are called *instance variables*. The code is contained within *methods*. Collectively, the methods and variables defined within a class are called *members* of the class. In most classes, the instance variables are acted upon and accessed by the methods defined for that class. Thus, as a general rule, it is the methods that determine how a class' data can be used.

Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another. We will come back to this point shortly, but it is an important concept to learn early.

All methods have the same general form as **main()**, which we have been using thus far. However, most methods will not be specified as **static** or **public**. Notice that the general form of a class does not specify a **main()** method. Java classes do not need to have a **main()** method. You only specify one if that class

is the starting point for your program. Further, some kinds of Java applications don't require a **main()** method at all.

A Simple Class

Let's begin our study of the class with a simple example. Here is a class called **Box** that defines three instance variables: **width**, **height**, and **depth**. Currently, **Box** does not contain any methods (but some will be added soon).

```
class Box {  
    double width;  
    double height;  
    double depth;  
}
```

As stated, a class defines a new type of data. In this case, the new data type is called **Box**. You will use this name to declare objects of type **Box**. It is important to remember that a class declaration only creates a template; it does not create an actual object. Thus, the preceding code does not cause any objects of type **Box** to come into existence.

To actually create a **Box** object, you will use a statement like the following:

```
Box mybox = new Box(); // create a Box object called mybox
```

After this statement executes, **mybox** will refer to an instance of **Box**. Thus, it will have "physical" reality. For the moment, don't worry about the details of this statement.

As mentioned earlier, each time you create an instance of a class, you are creating an object that contains its own copy of each instance variable defined by the class. Thus, every **Box** object will contain its own copies of the instance variables **width**, **height**, and **depth**. To access these variables, you will use the *dot* (.) operator. The dot operator links the name of the object with the name of an instance variable. For example, to assign the **width** variable of **mybox** the value 100, you would use the following statement:

```
mybox.width = 100;
```

This statement tells the compiler to assign the copy of **width** that is contained within the **mybox** object the value of 100. In general, you use the dot operator to access both the instance variables and the methods within an object. One other

point: Although commonly referred to as the dot *operator*, the formal specification for Java categorizes the . as a separator. However, since the use of the term “dot operator” is widespread, it is used in this book.

Here is a complete program that uses the **Box** class:

```
/* A program that uses the Box class.

    Call this file BoxDemo.java
*/
class Box {
    double width;
    double height;
    double depth;
}

// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;

        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;

        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;

        System.out.println("Volume is " + vol);
    }
}
```

You should call the file that contains this program **BoxDemo.java**, because the **main()** method is in the class called **BoxDemo**, not the class called **Box**. When you compile this program, you will find that two **.class** files have been created, one for **Box** and one for **BoxDemo**. The Java compiler automatically puts each class into its own **.class** file. It is not necessary for both the **Box** and the **BoxDemo** class to actually be in the same source file. You could put each class

in its own file, called **Box.java** and **BoxDemo.java**, respectively.

To run this program, you must execute **BoxDemo.class**. When you do, you will see the following output:

```
Volume is 3000.0
```

As stated earlier, each object has its own copies of the instance variables. This means that if you have two **Box** objects, each has its own copy of **depth**, **width**, and **height**. It is important to understand that changes to the instance variables of one object have no effect on the instance variables of another. For example, the following program declares two **Box** objects:

```
// This program declares two Box objects.
```

```
class Box {
    double width;
    double height;
    double depth;
}

class BoxDemo2 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // compute volume of first box
        vol = mybox1.width * mybox1.height * mybox1.depth;
        System.out.println("Volume is " + vol);

        // compute volume of second box
        vol = mybox2.width * mybox2.height * mybox2.depth;
        System.out.println("Volume is " + vol);
    }
}
```

The output produced by this program is shown here:

```
Volume is 3000.0
Volume is 162.0
```

As you can see, **mybox1**'s data is completely separate from the data contained in **mybox2**.

Declaring Objects

As just explained, when you create a class, you are creating a new data type. You can use this type to declare objects of that type. However, obtaining objects of a class is a two-step process. First, you must declare a variable of the class type.

This variable does not define an object. Instead, it is simply a variable that can *refer* to an object. Second, you must acquire an actual, physical copy of the object and assign it to that variable. You can do this using the **new** operator. The **new** operator dynamically allocates (that is, allocates at run time) memory for an object and returns a reference to it. This reference is, essentially, the address in memory of the object allocated by **new**. This reference is then stored in the variable. Thus, in Java, all class objects must be dynamically allocated. Let's look at the details of this procedure.

In the preceding sample programs, a line similar to the following is used to declare an object of type **Box**:

```
Box mybox = new Box();
```

This statement combines the two steps just described. It can be rewritten like this to show each step more clearly:

```
Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object
```

The first line declares **mybox** as a reference to an object of type **Box**. At this point, **mybox** does not yet refer to an actual object. The next line allocates an object and assigns a reference to it to **mybox**. After the second line executes, you can use **mybox** as if it were a **Box** object. But in reality, **mybox** simply holds, in essence, the memory address of the actual **Box** object. The effect of these two lines of code is depicted in [Figure 6-1](#).

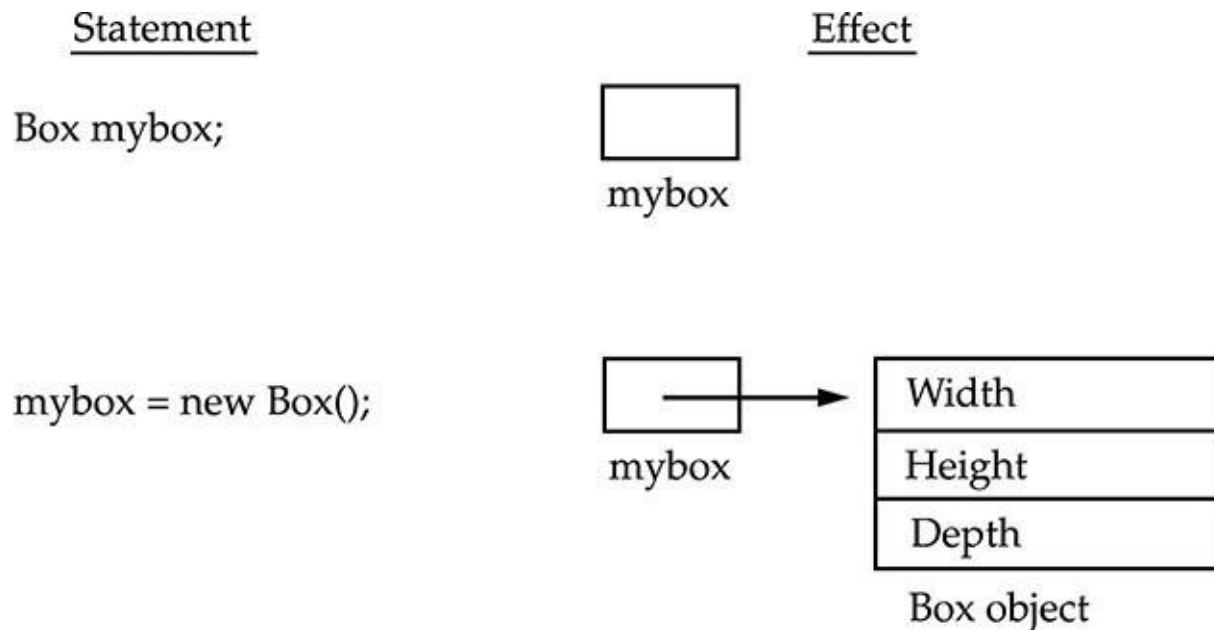


Figure 6-1 Declaring an object of type **Box**

A Closer Look at **new**

As just explained, the **new** operator dynamically allocates memory for an object. In the context of an assignment, it has this general form:

class-var = new *classname* ();

Here, *class-var* is a variable of the class type being created. The *classname* is the name of the class that is being instantiated. The class name followed by parentheses specifies the *constructor* for the class. A constructor defines what occurs when an object of a class is created. Constructors are an important part of all classes and have many significant attributes. Most real-world classes explicitly define their own constructors within their class definition. However, if no explicit constructor is specified, then Java will automatically supply a default constructor. This is the case with **Box**. For now, we will use the default constructor. Soon, you will see how to define your own constructors.

At this point, you might be wondering why you do not need to use **new** for such things as integers or characters. The answer is that Java's primitive types are not implemented as objects. Rather, they are implemented as "normal" variables. This is done in the interest of efficiency. As you will see, objects have many features and attributes that require Java to treat them differently than it treats the primitive types. By not applying the same overhead to the primitive types that applies to objects, Java can implement the primitive types more

efficiently. Later, you will see object versions of the primitive types that are available for your use in those situations in which complete objects of these types are needed.

It is important to understand that **new** allocates memory for an object during run time. The advantage of this approach is that your program can create as many or as few objects as it needs during the execution of your program.

However, since memory is finite, it is possible that **new** will not be able to allocate memory for an object because insufficient memory exists. If this happens, a run-time exception will occur. (You will learn how to handle exceptions in [Chapter 10](#).) For the sample programs in this book, you won't need to worry about running out of memory, but you will need to consider this possibility in real-world programs that you write.

Let's once again review the distinction between a class and an object. A class creates a new data type that can be used to create objects. That is, a class creates a logical framework that defines the relationship between its members. When you declare an object of a class, you are creating an instance of that class. Thus, a class is a logical construct. An object has physical reality. (That is, an object occupies space in memory.) It is important to keep this distinction clearly in mind.

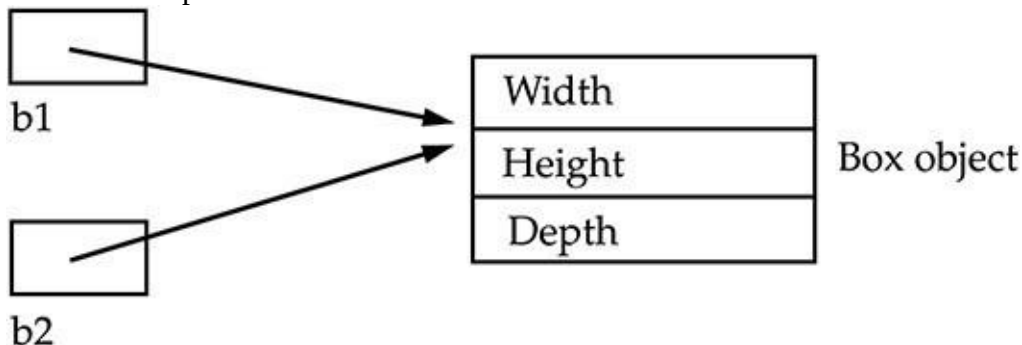
Assigning Object Reference Variables

Object reference variables act differently than you might expect when an assignment takes place. For example, what do you think the following fragment does?

```
Box b1 = new Box(); Box b2 = b1;
```

You might think that **b2** is being assigned a reference to a copy of the object referred to by **b1**. That is, you might think that **b1** and **b2** refer to separate and distinct objects. However, this would be wrong. Instead, after this fragment executes, **b1** and **b2** will both refer to the *same* object. The assignment of **b1** to **b2** did not allocate any memory or copy any part of the original object. It simply makes **b2** refer to the same object as does **b1**. Thus, any changes made to the object through **b2** will affect the object to which **b1** is referring, since they are the same object.

This situation is depicted here:



Although **b1** and **b2** both refer to the same object, they are not linked in any other way. For example, a subsequent assignment to **b1** will simply *unhook* **b1** from the original object without affecting the object or affecting **b2**. For example:

```
Box b1 = new Box(); Box b2 = b1;  
// ...  
b1 = null;
```

Here, **b1** has been set to **null**, but **b2** still points to the original object.

Introducing Methods

As mentioned at the beginning of this chapter, classes usually consist of two things: instance variables and methods. The topic of methods is a large one because Java gives them so much power and flexibility. In fact, much of the next chapter is devoted to methods. However, there are some fundamentals that you need to learn now so that you can begin to add methods to your classes.

This is the general form of a method:

```
type name(parameter-list) {  
    // body of method  
}
```

Here, *type* specifies the type of data returned by the method. This can be any valid type, including class types that you create. If the method does not return a value, its return type must be **void**. The name of the method is specified by *name*. This can be any legal identifier other than those already used by other items within the current scope. The *parameter-list* is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the arguments passed to the method when it is called. If the method has no parameters, then the parameter list will be empty.

Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:

```
return value;
```

Here, *value* is the value returned.

In the next few sections, you will see how to create various types of methods, including those that take parameters and those that return values.

Adding a Method to the Box Class

Although it is perfectly fine to create a class that contains only data, it rarely happens. Most of the time, you will use methods to access the instance variables defined by the class. In fact, methods define the interface to most classes. This allows the class implementor to hide the specific layout of internal data structures behind cleaner method abstractions. In addition to defining methods that provide access to data, you can also define methods that are used internally by the class itself.

Let's begin by adding a method to the **Box** class. It may have occurred to you while looking at the preceding programs that the computation of a box's volume was something that was best handled by the **Box** class rather than the **BoxDemo** class. After all, since the volume of a box is dependent upon the size of the box, it makes sense to have the **Box** class compute it. To do this, you must add a method to **Box**, as shown here:

```
// This program includes a method inside the box class.
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // display volume of a box  
    void volume() {  
        System.out.print("Volume is ");  
        System.out.println(width * height * depth);  
    }  
}  
  
class BoxDemo3 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
  
        // assign values to mybox1's instance variables  
        mybox1.width = 10;  
        mybox1.height = 20;  
        mybox1.depth = 15;  
  
        /* assign different values to mybox2's  
           instance variables */  
        mybox2.width = 3;  
        mybox2.height = 6;  
        mybox2.depth = 9;  
  
        // display volume of first box  
        mybox1.volume();  
  
        // display volume of second box  
        mybox2.volume();  
    }  
}
```


This program generates the following output, which is the same as the previous version.

```
Volume is 3000.0  
Volume is 162.0
```

Look closely at the following two lines of code:

```
mybox1.volume(); mybox2.volume();
```

The first line here invokes the **volume()** method on **mybox1**. That is, it calls **volume()** relative to the **mybox1** object, using the object's name followed by the dot operator. Thus, the call to **mybox1.volume()** displays the volume of the box defined by **mybox1**, and the call to **mybox2.volume()** displays the volume of the box defined by **mybox2**. Each time **volume()** is invoked, it displays the volume for the specified box.

If you are unfamiliar with the concept of calling a method, the following discussion will help clear things up. When **mybox1.volume()** is executed, the Java run-time system transfers control to the code defined inside **volume()**.

After the statements inside **volume()** have executed, control is returned to the calling routine, and execution resumes with the line of code following the call. In the most general sense, a method is Java's way of implementing subroutines.

There is something very important to notice inside the **volume()** method: the instance variables **width**, **height**, and **depth** are referred to directly, without preceding them with an object name or the dot operator. When a method uses an instance variable that is defined by its class, it does so directly, without explicit reference to an object and without use of the dot operator. This is easy to understand if you think about it. A method is always invoked relative to some object of its class. Once this invocation has occurred, the object is known. Thus, within a method, there is no need to specify the object a second time. This means that **width**, **height**, and **depth** inside **volume()** implicitly refer to the copies of those variables found in the object that invokes **volume()**.

Let's review: When an instance variable is accessed by code that is not part of the class in which that instance variable is defined, it must be done through an object, by use of the dot operator. However, when an instance variable is accessed by code that is part of the same class as the instance variable, that variable can be referred to directly. The same thing applies to methods.

Returning a Value

While the implementation of **volume()** does move the computation of a box's volume inside the **Box** class where it belongs, it is not the best way to do it. For example, what if another part of your program wanted to know the volume of a box, but not display its value? A better way to implement **volume()** is to have it compute the volume of the box and return the result to the caller. The following example, an improved version of the preceding program, does just that:

```

// Now, volume() returns the volume of a box.

class Box {
    double width;
    double height;
    double depth;

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

class BoxDemo4 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();
        double vol;

        // assign values to mybox1's instance variables
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;

        /* assign different values to mybox2's
           instance variables */
        mybox2.width = 3;
        mybox2.height = 6;
        mybox2.depth = 9;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}

```

As you can see, when **volume()** is called, it is put on the right side of an assignment statement. On the left is a variable, in this case **vol**, that will receive the value returned by **volume()**. Thus, after

```
vol = mybox1.volume();
```

executes, the value of **mybox1.volume()** is 3,000 and this value then is stored in **vol**.

There are two important things to understand about returning values:

- The type of data returned by a method must be compatible with the return type specified by the method. For example, if the return type of some method is **boolean**, you could not return an integer.
- The variable receiving the value returned by a method (such as **vol**, in this case) must also be compatible with the return type specified for the method.

One more point: The preceding program can be written a bit more efficiently because there is actually no need for the **vol** variable. The call to **volume()** could have been used in the **println()** statement directly, as shown here:

```
System.out.println("Volume is" + mybox1.volume());
```

In this case, when **println()** is executed, **mybox1.volume()** will be called automatically and its value will be passed to **println()**.

Adding a Method That Takes Parameters

While some methods don't need parameters, most do. Parameters allow a method to be generalized. That is, a parameterized method can operate on a variety of data and/or be used in a number of slightly different situations. To illustrate this point, let's use a very simple example. Here is a method that returns the square of the number 10:

```
int square()
{
    return 10 * 10;
}
```

While this method does, indeed, return the value of 10 squared, its use is very limited. However, if you modify the method so that it takes a parameter, as shown next, then you can make **square()** much more useful.

```
int square(int i)
{
    return i * i;
}
```

Now, **square()** will return the square of whatever value it is called with. That is, **square()** is now a general-purpose method that can compute the square of any integer value, rather than just 10.

Here is an example:

```
int x, y;  
x = square(5); // x equals 25 x = square(9); // x equals 81 y = 2;  
x = square(y); // x equals 4
```

In the first call to **square()**, the value 5 will be passed into parameter **i**. In the second call, **i** will receive the value 9. The third invocation passes the value of **y**, which is 2 in this example. As these examples show, **square()** is able to return the square of whatever data it is passed.

It is important to keep the two terms *parameter* and *argument* straight. A *parameter* is a variable defined by a method that receives a value when the method is called. For example, in **square()**, **i** is a parameter. An *argument* is a value that is passed to a method when it is invoked. For example, **square(100)** passes 100 as an argument. Inside **square()**, the parameter **i** receives that value.

You can use a parameterized method to improve the **Box** class. In the preceding examples, the dimensions of each box had to be set separately by use of a sequence of statements, such as:

```
mybox1.width = 10;  
mybox1.height = 20;  
mybox1.depth = 15;
```

While this code works, it is troubling for two reasons. First, it is clumsy and error prone. For example, it would be easy to forget to set a dimension. Second, in well-designed Java programs, instance variables should be accessed only through methods defined by their class. In the future, you can change the behavior of a method, but you can't change the behavior of an exposed instance variable.

Thus, a better approach to setting the dimensions of a box is to create a method that takes the dimensions of a box in its parameters and sets each instance variable appropriately. This concept is implemented by the following program:

```
// This program uses a parameterized method.
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
  
    // sets dimensions of box  
    void setDim(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
}
```

```
class BoxDemo5 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
  
        // initialize each box  
        mybox1.setDim(10, 20, 15);  
        mybox2.setDim(3, 6, 9);  
  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

As you can see, the **setDim()** method is used to set the dimensions of each box. For example, when

```
mybox1.setDim(10, 20, 15);
```

is executed, 10 is copied into parameter **w**, 20 is copied into **h**, and 15 is copied into **d**. Inside **setDim()** the values of **w**, **h**, and **d** are then assigned to **width**, **height**, and **depth**, respectively.

For many readers, the concepts presented in the preceding sections will be familiar. However, if such things as method calls, arguments, and parameters are new to you, then you might want to take some time to experiment before moving on. The concepts of the method invocation, parameters, and return values are fundamental to Java programming.

Constructors

It can be tedious to initialize all of the variables in a class each time an instance is created. Even when you add convenience methods like **setDim()**, it would be simpler and more concise to have all of the setup done at the time the object is first created. Because the requirement for initialization is so common, Java allows objects to initialize themselves when they are created. This automatic initialization is performed through the use of a constructor.

A constructor initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method.

Once defined, the constructor is automatically called when the object is created, before the **new** operator completes. Constructors look a little strange because they have no return type, not even **void**. This is because the implicit return type of a class' constructor is the class type itself. It is the constructor's job to initialize the internal state of an object so that the code creating an instance will have a fully initialized, usable object immediately.

You can rework the **Box** example so that the dimensions of a box are automatically initialized when an object is constructed. To do so, replace **setDim()** with a constructor. Let's begin by defining a simple constructor that sets the dimensions of each box to the same values. This version is shown here:

```

/* Here, Box uses a constructor to initialize the
   dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box() {
        System.out.println("Constructing Box");
        width = 10;
        height = 10;
        depth = 10;
    }

    // compute and return volume
    double volume() {
        return width * height * depth;
    }
}

class BoxDemo6 {
    public static void main(String args[]) {
        // declare, allocate, and initialize Box objects
        Box mybox1 = new Box();
        Box mybox2 = new Box();

        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}

```

When this program is run, it generates the following results:

```
Constructing Box Constructing Box Volume is 1000.0
Volume is 1000.0
```

As you can see, both **mybox1** and **mybox2** were initialized by the **Box()** constructor when they were created. Since the constructor gives all boxes the same dimensions, 10 by 10 by 10, both **mybox1** and **mybox2** will have the same volume. The **println()** statement inside **Box()** is for the sake of illustration only. Most constructors will not display anything. They will simply initialize an object.

Before moving on, let's reexamine the **new** operator. As you know, when you allocate an object, you use the following general form:

```
class-var = new classname ( );
```

Now you can understand why the parentheses are needed after the class name. What is actually happening is that the constructor for the class is being called. Thus, in the line

```
Box mybox1 = new Box( );
```

new Box() is calling the **Box()** constructor. When you do not explicitly define a constructor for a class, then Java creates a default constructor for the class. This is why the preceding line of code worked in earlier versions of **Box** that did not define a constructor. When using the default constructor, all non-initialized instance variables will have their default values, which are zero, **null**, and **false**, for numeric types, reference types, and **boolean**, respectively. The default constructor is often sufficient for simple classes, but it usually won't do for more sophisticated ones. Once you define your own constructor, the default constructor is no longer used.

Parameterized Constructors

While the **Box()** constructor in the preceding example does initialize a **Box** object, it is not very useful—all boxes have the same dimensions. What is needed is a way to construct **Box** objects of various dimensions. The easy solution is to add parameters to the constructor. As you can probably guess, this makes it much more useful. For example, the following version of **Box** defines a

parameterized constructor that sets the dimensions of a box as specified by those parameters. Pay special attention to how **Box** objects are created.

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
    double width;
    double height;
    double depth;

    // This is the constructor for Box.
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // compute and return volume
    double volume() {

        return width * height * depth;
    }
}

class BoxDemo7 {
    public static void main(String args[]) {
        // declare, allocate, and initialize Box objects
        Box mybox1 = new Box(10, 20, 15);
        Box mybox2 = new Box(3, 6, 9);

        double vol;

        // get volume of first box
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);

        // get volume of second box
        vol = mybox2.volume();
        System.out.println("Volume is " + vol);
    }
}
```

The output from this program is shown here:

```
Volume is 3000.0  
Volume is 162.0
```

As you can see, each object is initialized as specified in the parameters to its constructor. For example, in the following line,

```
Box mybox1 = new Box(10, 20, 15);
```

the values 10, 20, and 15 are passed to the **Box()** constructor when **new** creates the object. Thus, **mybox1**'s copy of **width**, **height**, and **depth** will contain the values 10, 20, and 15, respectively.

The this Keyword

Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the **this** keyword. **this** can be used inside any method to refer to the *current* object. That is, **this** is always a reference to the object on which the method was invoked. You can use **this** anywhere a reference to an object of the current class' type is permitted.

To better understand what **this** refers to, consider the following version of **Box()**:

```
// A redundant use of this.  
Box(double w, double h, double d) {  
    this.width = w;  
    this.height = h;  
    this.depth = d;  
}
```

This version of **Box()** operates exactly like the earlier version. The use of **this** is redundant, but perfectly correct. Inside **Box()**, **this** will always refer to the invoking object. While it is redundant in this case, **this** is useful in other contexts, one of which is explained in the next section.

Instance Variable Hiding

As you know, it is illegal in Java to declare two local variables with the same name inside the same or enclosing scopes. Interestingly, you can have local

variables, including formal parameters to methods, which overlap with the names of the class' instance variables. However, when a local variable has the same name as an instance variable, the local variable *hides* the instance variable. This is why **width**, **height**, and **depth** were not used as the names of the parameters to the **Box()** constructor inside the **Box** class. If they had been, then **width**, for example, would have referred to the formal parameter, hiding the instance variable **width**. While it is usually easier to simply use different names, there is another way around this situation. Because **this** lets you refer directly to the object, you can use it to resolve any namespace collisions that might occur between instance variables and local variables. For example, here is another version of **Box()**, which uses **width**, **height**, and **depth** for parameter names and then uses **this** to access the instance variables by the same name:

```
// Use this to resolve name-space collisions.
Box(double width, double height, double depth) {
    this.width = width;
    this.height = height;
    this.depth = depth;
}
```

A word of caution: The use of **this** in such a context can sometimes be confusing, and some programmers are careful not to use local variables and formal parameter names that hide instance variables. Of course, other programmers believe the contrary—that it is a good convention to use the same names for clarity, and use **this** to overcome the instance variable hiding. It is a matter of taste which approach you adopt.

Garbage Collection

Since objects are dynamically allocated by using the **new** operator, you might be wondering how such objects are destroyed and their memory released for later reallocation. In some languages, such as traditional C++, dynamically allocated objects must be manually released by use of a **delete** operator. Java takes a different approach; it handles deallocation for you automatically. The technique that accomplishes this is called *garbage collection*. It works like this: when no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed. There is no need to explicitly destroy objects. Garbage collection only occurs sporadically (if at all) during the execution of your program. It will not occur simply because one or

more objects exist that are no longer used. Furthermore, different Java run-time implementations will take varying approaches to garbage collection, but for the most part, you should not have to think about it while writing your programs.

V. Static vs. Non static Members

Static and non-static members Java static keyword the static keyword in Java is used for memory management mainly. We can apply java static keyword with variables, methods, blocks and nested class. The static keyword belongs to the class than an instance of the class. JAVA Static variable: If you declare any variable as static, it is known as a static variable. The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc. It makes your program memory efficient (i.e., it saves memory).

Static variables

```
class Student
```

```
{ int rollno; String name; String college="ITS"; }
```

Suppose there are 500 students in my college, now all instance data members will get memory each time when the object is created. All students have its unique rollno and name, so instance data member is good in such case. Here, "college" refers to the common property of all objects. If we make it static, this field will get the memory only once.

Static variables:

//Java Program to demonstrate the use of static variable

```
class Student{
```

```
int rollno;//instance variable
```

```
String name;
```

```
static String college ="ITS";//static variable
```

```
//constructor
```

```
Student(int r, String n){
```

```
rollno = r;
```

```
name = n;
```

```
}
```

```
//method to display the values
```

```
void display (){System.out.println(rollno+" "+name+" "+college);}

}
```

Output:

111 Karan ITS

222 Aryan ITS

Static methods:

If we apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Java program to call a static method

```
class Test {

// static method

public static int sum(int a, int b)

{

return a + b;

}

}

class Main {

public static void main(String[] args)

{ int n = 3, m = 6;

// call the static method

int s = Test.sum(n, m);

System.out.print("sum is = " + s);

}}
```

Output:

sum is = 9

VII. Control Structures recap

Control Statements

Programming languages use control statements to cause the flow of execution to advance and branch based on changes to the state of a program.

Categories of Java's Control statements

Java's program control statements can be put into **three categories**:

- 1) selection,
- 2) iteration, and
- 3) jump.

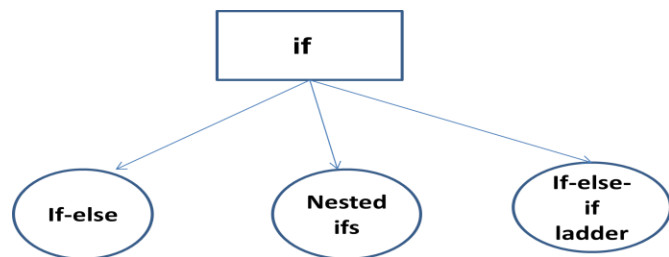
1) Selection statements (if and switch)

Selection statements allow the program to choose different paths of execution based upon the outcome of an expression or the state of a variable.

Java supports two selection statements: **if** and **switch**.

if

- The **if** statement is Java's conditional branch statement.
- It can be used to route program execution through two different paths.



General form

```
if (condition) statement1;  
else statement2;
```

- Each statement may be a single statement or a compound statement enclosed in curly braces (that is, a block).
- The condition is any expression that returns a boolean value.
- **else** clause is optional.

Working of if

- If the *condition* is true, then *statement1* is executed.
- Otherwise, *statement2* (if it exists) is executed.

Example:

```
int a, b;  
if(a < b) a = 0;  
else b = 0;
```

Explanation:

If a is less than b, then a is set to zero. Otherwise, b is set to zero.

Nested ifs

A nested if is an if statement that is the target of another if or else.

Example:

```
if(i == 10) {  
    if(j < 20) a = b;  
    if(k > 100) c = d;
```

```

        else a = c;
    }
    else a = d;

```

The if-else-if Ladder

A sequence of nested ifs is the if-else-if ladder.

```

if(condition)
    statement;
else if(condition)
    statement;
else if(condition)
    statement;
.
.
.
else
    statement;

```

Working of if-else-if ladder

- 1) The if statements are executed from the top down.
- 2) As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the ladder is bypassed.
- 3) If none of the conditions is true, then the final else statement will be executed.
- 4) If there is no final else and all other conditions are false, then no action will take place.

// Demonstration if-else-if statements.

```

class IfElse {
    public static void main(String args[]) {
        int month = 4; // April
        String season;
        if(month == 12 || month == 1 || month == 2)
            season = "Winter";
        else if(month == 3 || month == 4 || month == 5)
            season = "Spring";
        else if(month == 6 || month == 7 || month == 8)
            season = "Summer";
        else if(month == 9 || month == 10 || month == 11)
            season = "Autumn";
        else
            season = "Invalid Month";

        System.out.println("April is in the " + season + ".");
    }
}

```

Q: This program is to determine which season a particular month is in.

Output:

April is in the Spring.

switch

```

switch (expression) {
    case value1:
        // statement sequence
        break;
    case value2:
        // statement sequence

```

- The **switch** statement is Java's multiway branch statement.
- It provides an easy way to dispatch execution to different parts of your code based on the value of an expression.
- It often provides a better alternative than a large series of if-else-if statements.
- *expression* must be of type **byte, short, int, char, enum, or String**
- Duplicate **case** values are not allowed. **case** constants must be unique.


```

        break;
    .
    .
    .
case valueN :
    // statement sequence
    break;
default:
    // default statement sequence
}

```

Working of switch

- The value of the **expression** is compared with each of the values in the case statements.
- If a match is found, the code sequence following that case statement is executed.
- If none of the constants matches the value of the expression, then the **default** statement is executed.
- However, the **default** statement is optional.
- If no case matches and no default is present, then no further action is taken.
- The **break** statement is used inside the switch to terminate a statement sequence.
- When a **break** statement is encountered, execution branches to the first line of code that follows the entire switch statement.
- **break** is also optional. If **break** is omitted, the program just keeps executing the remaining case blocks until either a break is found or the switch statement ends.

//Demonstration of switch statement

```

class Switch {
    public static void main(String args[]) {
        int month = 4;
        String season;

        switch (month) {
            case 12:
            case 1:
            case 2:
                season = "Winter";
                break;
            case 3:
            case 4:
            case 5:
                season = "Spring";
                break;
            case 6:
            case 7:
            case 8:
                season = "Summer";
                break;
            case 9:

```

Q: This program is to determine which season a particular month is in.

Output:

April is in the Spring.

```

        case 10:
        case 11:
            season = "Autumn";
            break;
        default:
            season = "Invalid Month";
    }
    System.out.println("April is in the " + season + ".");
}
}

```

Switch Example without break:

```

int x = 1;
switch(x) {
    case 1: System.out.println("x is one");
    case 2: System.out.println("x is two");
    case 3: System.out.println("x is three");
}
System.out.println("out of the switch");

```

Output:

```

x is one
x is two
x is three
out of the switch

```

Nested switch Statements

We can write a switch as part of the statement sequence of an outer switch, this is called a *nested* switch.

Since a switch statement defines its own block, **no conflicts** arise between the case constants in the **inner switch** and those in the **outer switch**.

```

switch(count) {
    case 1:
        switch(target) { // nested switch
            case 1: // no conflicts with outer switch
                System.out.println("target is one");
                break;
        }
        break;
    case 2: // ...

```

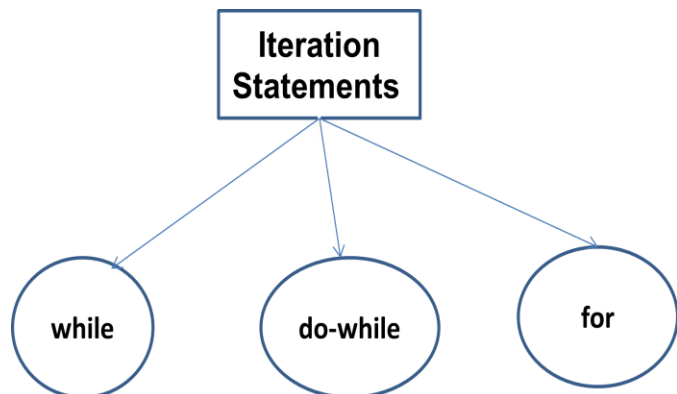
Here, outer switch and inner switch contains the same case constant 1, but it is perfectly valid.

Difference between if and switch

- The **switch** differs from the **if** in that switch can only test for equality, whereas if can evaluate any type of Boolean expression.
- A **switch** statement is usually more efficient than a set of **nested ifs**.
- If we need to select among a large group of values, a **switch** statement will run much faster than the equivalent logic coded using a sequence of **if-elses**.

Iteration statements (while,do-while and for)

- Java's **iteration** statements are **for**, **while**, and **do-while**
- Iteration statements enable program execution to repeat one or more statements, which are called as **loops**.
- A **loop** repeatedly executes the same set of instructions until a termination condition is met.



while

It repeats a statement or block while its controlling expression is true.

syntax

```
while(condition)
{
    // body of loop
}
```

Working of while

- The condition can be any Boolean expression.
- The body of the loop will be executed as long as the conditional expression is true.
- When condition becomes false, control passes to the next line of code immediately following the loop.
- The curly braces are unnecessary if only a single statement is being repeated.

Example

```
class While {
    public static void main(String args[]) {
        int n = 1;
        while(n < 5) {
            System.out.println(n);
            n++;
        }
    }
}
```

Output

```
1
2
3
4
```

Null statement

null statement is a statement which consists only of a semicolon.

The body of the while can be empty, that is, it can be a **null statement**.

Example

```
int n = 1;
while(n++ < 5);
```

Output:

No output will be printed but the loop stops execution after 4 iterations.

do-while

```
do {
    // body of loop
} while (condition);
```

Working of do-while

- In **do-while**, conditional expression is at the bottom of the loop.
- Each iteration of the do-while loop first executes the body of the loop and then evaluates the conditional expression.
- If this expression is true, the loop will repeat. Otherwise, the loop terminates.
- The do-while loop always executes its body at least once.

Example

```
class While {
    public static void main(String args[]) {
        int n = 1;
        do{
            System.out.println(n);
            n++;
        }while(n < 5);
    }
}
```

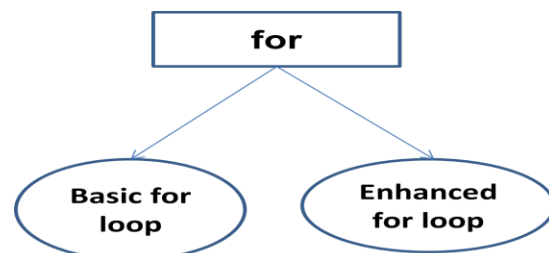
Output

```
1
2
3
4
```

for

Beginning with JDK 5, there are two forms of the for loop.

1. **Basic for loop**
2. **For-each (Enhanced for loop)**



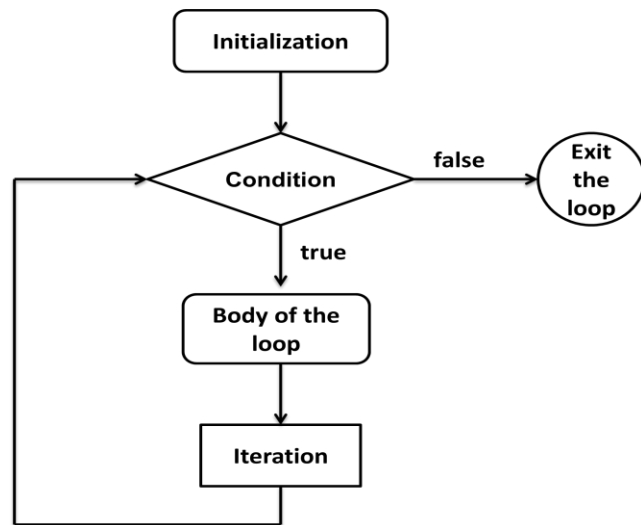
Basic for loop

```
for(initialization; condition; iteration)
{
    // body
}
```

Working of for loop

1. When the loop first starts, the **initialization** portion of the loop is executed.
2. Next, **condition** is evaluated. This must be a Boolean expression.
3. If this expression is true, then the body of the loop is executed. If it is false, the loop terminates.
4. Next, the **iteration** portion of the loop is executed.
5. The loop then iterates, first evaluating the conditional expression, then executing the body of the loop, and then executing the iteration expression with each pass.

Flow Diagram



Example

```
class For {
    public static void main(String args[]) {
        for(i=1; i<5; i++) {
            System.out.println(n);
        }
    }
}
```

Output

```
1
2
3
4
```

1. Declaring Loop Control Variables Inside the for Loop

It is possible to declare the variable inside the initialization portion of the for.

```
// here, n is declared inside of the for loop
for(int n=10; n>0; n--)
    System.out.println(n);
```

2. Using the Comma

- i. Using **comma**, we can include more than one statement in the **initialization** and **iteration** portions of the **for** loop.
- ii. We can not use more than one statement in increment section.

```
int a, b;

for(a=1, b=4; a<b; a++, b--) {
    System.out.println("a = " + a);
    System.out.println("b = " + b);
}
```

Output:

```
a=1
b=4
a=2
b=3
```

For loop variations

1. The condition can be any Boolean expression.
2. Either the **initialization** or the **iteration** expression or both may be absent.

```
public static void main(String args[]) {
    int i;
    boolean done = false;

    i = 0;
    for( ; !done; ) {
        System.out.println( i);
        if(i == 4) done = true;
        i++;
    }
}
```

Output:

```
1
2
3
4
```

3. We can leave all three parts of the **for** empty.

```
for( ; ; ) {
    // ...
}
```

Output:
Infinite loop

The For-Each Version of the for Loop

- i. Beginning with JDK 5, a second form of **for** was defined that implements a “for-each” style loop.
- ii. A **for-each** style loop is designed to cycle through a collection of objects, such as an array, in strictly sequential fashion, from start to finish.

form of for-each

```
for(type itr-var : collection)
    statement-block
```

Here,

- **type** specifies the type

- **itr-var** specifies the name of an iteration variable that will receive the elements from a collection, one at a time, from beginning to end.
- With each iteration of the loop, the next element in the collection is retrieved and stored in itr-var.
- The loop repeats until all elements in the collection have been obtained.

Example 1:

```
int nums[] = { 1, 2, 3, 4, 5 };

// use for-each style for to display and sum the values
for(int x : nums) {
    System.out.println("Value is: " + x);
}
```

Output:

```
Value is: 1
Value is: 2
Value is: 3
Value is: 4
Value is: 5
```

Example 2: Iterating over multidimensional arrays

```
int sum = 0;
int nums[][] = { {1,2},{3,4}};

// use for-each for to display and sum the values
for(int x[] : nums) {
    for(int y : x) {
        System.out.println("Value is: " + y);
        sum += y;
    }
}
System.out.println("Sum: " + sum);
```

Output:

```
Value is: 1
Value is: 2
Value is: 3
Value is: 4
Sum: 10
```

Nested For Loops

Like all other programming languages, Java allows loops to be nested. That is, one loop may be inside another.

// Loops may be nested.

```
class Nested {
    public static void main(String args[]) {
        int i, j;
        for(i=0; i<10; i++) {
            for(j=i; j<10; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Output:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * *
* * *
* **
* *
```

Jump Statements (break, continue, return)

Jump statements allow the program to execute in a nonlinear fashion.

Java supports three jump statements: **break**, **continue**, and **return**.

break

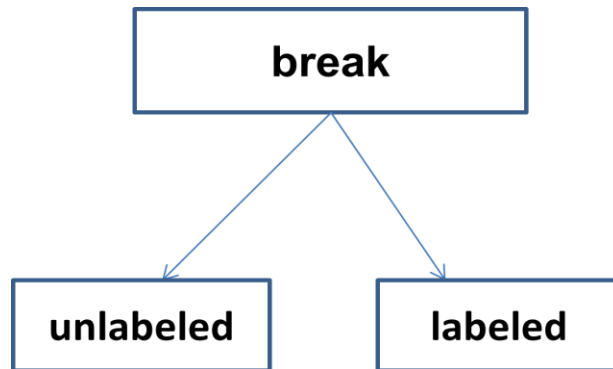
In Java, the break statement has **three** uses.

Unlabeled break

- 1) It terminates a statement sequence in a **switch** statement.
- 2) It can be used to **exit a loop**.

Labeled break:

- 3) It can be used as a “civilized” form of goto.



Unlabeled break : break to Exit a loop

- By using **break**, we can force immediate termination of a loop, bypassing the conditional expression and any remaining code in the body of the loop.
- When a **break** statement is encountered inside a loop, the loop is terminated and program control resumes at the next statement following the loop.
- The **break** statement can be used with any of Java's loops, including intentionally infinite loops.

Example

```
class BrkEx
{
    public static void main(String args[]) {
        for(int i=1; i<5;i++) {
            if(i == 4)
                break;
            System.out.println(n);
        }
    }
}
```

Output

```
1
2
3
```


Labeled break : Form of Goto

- In Java, **break** statement can also be used as **form of Goto**.
- Java does not have a goto statement. **break** gives you the benefits of a **goto** without its problems.

General form

break label;

- ✓ **label** is the name of a label that identifies a block of code.
- A **label** is any valid Java identifier followed by a colon.
- To name a block, put a label at the start of it.
- **Labeled break** causes execution to resume at the end of the **labeled block**.

Example:

```
class Break {
    public static void main(String args[])
    {
        boolean t = true;

        first:
        {
            second:
            {
                third:
                {
                    System.out.println("Before the break.");
                    if(t) break second; // break second block
                    System.out.println("This won't execute");
                }
                System.out.println("This won't execute");
            }
            System.out.println("This is after second block.");
        }
    }
}
```

Output:

Before the break.
This is after second block.

continue

- 1) **continue** is used to force an early iteration of the loop.
- 2) In any iteration, **continue** will stop processing the code that is present after the **continue** and continues the execution from the next iteration.
- 3) In **while** and **do-while** loops, a **continue** statement causes control to be transferred directly to the conditional expression that controls the loop.
- 4) In a **for** loop, control goes first to the iteration portion of the for statement and then to the conditional expression.

Example:

```
class Continue
{
    public static void main(String args[]) {
        for(int i=1; i<5;i++) {
            if( i % 2 == 0) continue;
            System.out.print(i);
        }
    }
}
```

Output

1
3

return

- The **return** statement is used to explicitly return from a method.
- It causes program control to transfer back to the caller of the method.
- The **return** statement immediately terminates the method in which it is executed.

Example

// Demonstrate return.

```
class Return {  
    public static void main(String args[]) {  
        boolean t = true;  
  
        System.out.println("Before the return.");  
  
        if(t) return; // return to caller  
  
        System.out.println("This won't execute.");  
    }  
}
```